RoomRecon: High-Quality Textured Room Layout Reconstruction on Mobile Devices

Seok Joon Kim* MAXST Co., Ltd.
Dinh Duc Cao† MAXST Co., Ltd.
Federica Spinola‡ Independent Researcher
Se Jin Lee* MAXST Co., Ltd.
Kyu Sung Cho* MAXST Co., Ltd.

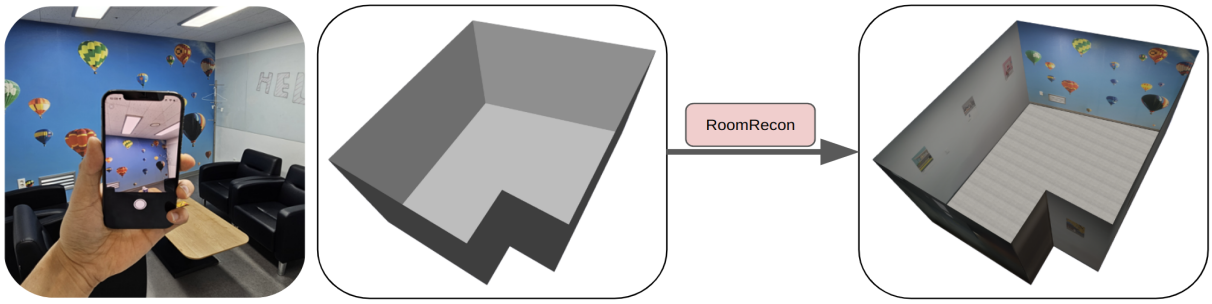


Figure 1: **Creating realistic textured 3D room models on mobile devices**: Capitalizing on the wide availability of mobile devices, RoomRecon creates textured 3D replicas of indoor spaces, readily applicable for use in VR and AR applications.

Abstract

Widespread RGB-Depth (RGB-D) sensors and advanced 3D reconstruction technologies facilitate the capture of indoor spaces, improving the fields of augmented reality (AR), virtual reality (VR), and extended reality (XR). Nevertheless, current technologies still face limitations, such as the inability to reflect minor scene changes without a complete recapture, the lack of semantic scene understanding, and various texturing challenges that affect the 3D model's visual quality. These issues affect the realism required for VR experiences and other applications such as in interior design and real estate. To address these challenges, we introduce RoomRecon, an interactive, real-time scanning and texturing pipeline for 3D room models. We propose a two-phase texturing pipeline that integrates AR-guided image capturing for texturing and generative AI models to improve texturing quality and provide better replicas of indoor spaces. Moreover, we suggest to focus only on permanent room elements such as walls, floors, and ceilings, to allow for easily customizable 3D models. We conduct experiments in a variety of indoor spaces to assess the texturing quality and speed of our method. The quantitative results and user study demonstrate that RoomRecon surpasses state-of-the-art methods in terms of texturing quality and on-device computation time.

Index Terms: Room layout, Indoor 3D reconstruction, Texturing, AR-assisted image capturing, Mobile application.

1 Introduction

Widespread RGB-Depth (RGB-D) sensors and improved 3D reconstruction methods [12, 28, 41] allow to better capture indoor spaces, easing the creation of augmented reality (AR), virtual reality (VR), and extended reality (XR) applications [10]. The recent integration of these 3D reconstruction technologies into mobile devices allows individuals to quickly and effortlessly capture their own spaces [4, 21, 44]. In fact, accurate replicas of scenes are sought after in fields like interior design and architecture [30, 40], where precise representations facilitate visualization, planning, and effective communication between coworkers. Additionally, such replicas offer benefits in real estate, enabling virtual property tours for potential buyers.

However, despite these technological advancements, several limitations persist in constructing 3D models. Firstly, the captured and reconstructed scenes are static, requiring users to fully recapture them when internal changes occur such as furniture rearrangements or new wall decorations. Secondly, current algorithms lack a semantic understanding of the scene, preventing selective geometry alterations. For instance, users cannot decide to remove the object *table* from their room's 3D model, nor can they choose to keep the object *bed*. Thirdly, environmental and technological challenges disrupt the texturing process, which is crucial for enhancing the realism of 3D models and improving user experience. The environmental constraints, such as obstructions by furniture, lead to untextured areas in the 3D models. Technological limitations, including blurred images and camera pose drift, produce textured outputs with blurs, seams, and lighting inconsistencies that disrupt visual coherence. These discrepancies are common problems of state-of-art (SOTA) texturing methods [7, 15, 37]. All of these issues affect the usability of 3D models in VR or AR applications, which require realistic, complete and dynamic virtual environments for immersive user experiences.

With the growing popularity of diffusion models, several noteworthy methods such as Text2Room [22], ControlRoom3D [33] and RoomDreamer [34] can produce aesthetically pleasing indoor spaces. From fully generated to more controlled 3D models, these methods can create many environments suitable for VR applications. Nevertheless, none of these methods tackle the problem of creating accurate virtual replicas of indoor spaces.

To address the challenges of accurately replicating indoor spaces with high realism and completeness, we introduce RoomRecon, an interactive, real-time 3D reconstruction and texturing mobile application for 3D room modeling. Firstly, RoomRecon proposes to capture only permanent room elements such as walls, floors, and ceilings (i.e. the room's layout). This reduces the need for recaptures as soon as furniture changes, allowing for durable and customizable representations of indoor spaces. Whilst it is beyond the scope of this work, we note that dynamic objects can be added

*Affiliated with MAXST Co., Ltd.
†Equal contribution.
‡e-mail: spinolafederica@yahoo.com.

to our 3D models by using existing CAD model databases [6, 26] and pre-captured textured meshes [14]. Secondly, RoomRecon proposes to selectively capture a small number of high-quality images to be used in the texturing phase, through a dedicated user interface (UI) and AR cues during the scanning process. This aims to improve texturing quality by ensuring that surfaces are textured by as few images as possible, helping to reduce blur, seams and structural inconsistencies. By addressing the texturing's environmental and technological challenges, RoomRecon aims to model 3D scenes that are accurate replicas of indoor spaces, distinguishing our method from previous solutions [22, 33, 34].

To summarise, our contributions are as follows:

1. We present RoomRecon, a mobile framework that efficiently generates complete and high-fidelity 3D models of rooms.
2. We present a divide-and-conquer strategy for surface texturing, relying on AR-guided human involvement.
3. We introduce the Plane2Image rendering module, which enables editing textured surfaces in 2D image space by using common image processing techniques or more advanced diffusion-based generative models.
4. We conduct extensive experiments and user studies on a variety of indoor spaces, achieving superior results both quantitatively and qualitatively compared to SOTA texturing methods.

2 Background and Related Work

2.1 Texturing for 3D Reconstruction

Texturing aims to ensure sharp and realistic appearances of 3D models to enhance user experience. As Zhao et al. [45] outlines, texturing methods can be categorized into four main types: average-based, warping-based, learning-based, and assignment-based. The most relevant for our work is the assignment-based method, which assigns specific images to each face of the model using a Markov Random Field (MRF) based optimization. Assignment-based methods utilize patches of original high-resolution images, leading to very sharp texturing results. Building on this foundation, Waechter et al. [37], Fu et al. [15, 16] and Zhao et al. [45] apply pairwise costs in the MRF optimization to enforce the texturing of neighboring faces with the same image, ensuring local consistency and sharpness. Taking the work of Waechter et al. [37] further, Fu et al. [15] introduce detail-aware texture chart partitions to avoid creating partitions in areas of the mesh with high detail, thus preserving texture integrity in these detailed regions. In contrast, average-based approaches often produce blurry textures or ghosting effects due to the process of averaging color values from multiple images. Zhou et al. [46] assign colors to mesh vertices by averaging the colors from all images capturing each vertex, a technique highly dependent on the mesh's resolution and generally requiring a very fine mesh to achieve satisfactory quality. Wang et al. [38] propose using 3D texels positioned within mesh faces. In the domain of learning-based texturing, Zhao et al. [45] and Huang et al. [23] leverage generative adversarial networks (GANs) [20] that are trained to differentiate between original and rendered images, thereby refining the texturing process. Finally, the work of Zhou et al. [46] and Fu et al. [16] stand out for their use of warping-based methods to achieve more accurate texture mappings. Given the hardware limitations of mobile devices and the benefits in speed and textured output clarity, we have adopted assignment-based texturing methods [15, 37] as the foundation for our approach.

2.2 Image Inpainting

Image inpainting is a process that repairs, erases, and fills data within masked areas of an image to restore or create content that blends with surrounding areas, rendering changes imperceptible to casual observers [31]. Whilst simple methods relying on image interpolation [24] can provide adequate results in some cases, recent learning-based methods [13, 25, 35] show better performance across a variety of applications. LAMA [35] proposes an inpainting network based on Fast Fourier Convolutions (FFCs) that create large receptive field, capturing the global context of images. MAT [25] presents a novel transformer-based model for large hole inpainting in high-resolution images. Attention mechanisms can learn better long range dependencies for structure recovery, but are slow, especially for large image sizes. ZIT [13] addresses these issues by proposing an additional structure restorer that facilitates incremental image inpainting. The recent development of diffusion models provides the ability to generate detailed images from text prompts [1, 29, 32, 47]. Its applications extend to many generative functions including image inpainting. In this paper, we employ ZIT [13] to correct texturing defects and fill untextured regions.

3 System Design

3.1 Problem Statement

Our work aims to achieve a complete and precise textured room layout that closely replicates the appearance of rooms. However, the texturing process faces complications inherent to capturing indoor spaces and stemming from the texturing methods used. Environmental challenges include occlusions by furniture that result in untextured areas, shadows cast by objects affecting wall textures, and difficulties in accurately capturing thin objects (e.g.: clothes hangers, chair legs), often leading to incomplete meshes. These problems are particularly present for floor areas and lower regions of the wall, affecting their texture. Ceilings pose texturing challenges due to uncomfortable image capture positions and limited mobility caused by furniture, resulting in images often taken at slanted angles. This can lead to textures with lower quality due to distortions and misalignments. Additionally, the repetitive patterns on ceilings and floors, characterized by numerous lines between samples, complicate the texturing process. In fact, even slight deviations in camera pose can lead to misaligned lines, resulting in noticeable errors in the final textured output. Finally, texturing transparent or reflective surfaces, such as glass walls and mirrors, can also be challenging. Using multiple images to texture such planar structures often results in viewpoint inconsistencies, as each image captures a different external scene and undesired reflections.

Technological problems associated with texturing algorithms are mostly observed in assignment-based methods such as MVS-Tex [37] and SeamlessTex [15]. While these methods can generate photorealistic textures from high-resolution images, they are prone to artifacts stemming from geometric inaccuracies and camera pose drift. These issues become particularly visible when multiple images are used to texture a single wall, often resulting in noticeable seams. For more details regarding environmental and technological issues, refer to the supplementary materials (Section A.4).

3.2 Overview

Our system aims to reconstruct rooms with high realism through a two-phase pipeline, as shown in Fig. 2. The first phase involves capturing an indoor space using an RGB-D sensor, such as an iPhone, to collect RGB images, depth maps, camera parameters, and the original mesh (M_{original}), described in Sec. 3.3. A standard algorithm then generates a 3D room layout, detailing plane normals, wall corners, and floor and ceiling heights, as referenced in various studies [5, 6, 9, 17]. This layout, along with M_{original}, is processed to generate a composite mesh (Sec. 3.4). The Texture Images Acquisition module captures a small set of high-quality images for texturing (Sec. 3.5). The second phase starts with the Texturing step, producing a fully textured 3D model (Sec. 3.6). Each textured plane is converted into an image via the Plane2Image module (Sec. 3.7), followed by texture refinement in the Post-Texturing stage, which includes on-device and cloud-based enhancements (Sec. 3.8). Ultimately, our pipeline delivers precise 3D replicas of indoor spaces.

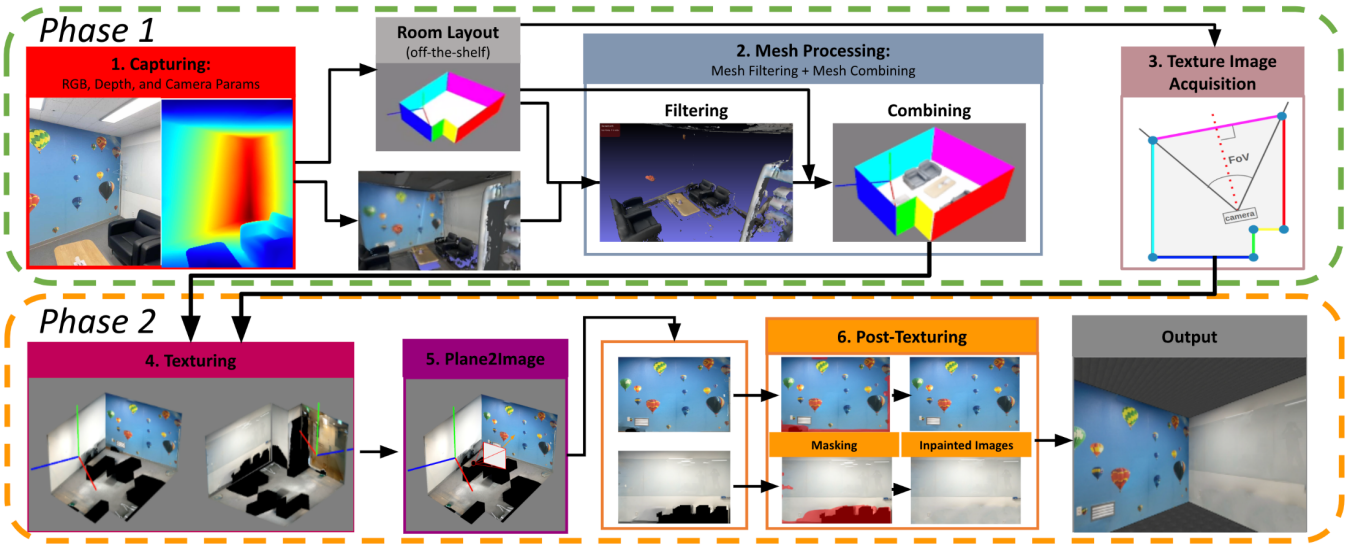


Figure 2: **Method overview**: Our system constructs detailed textured 3D room models through a two-phase pipeline. Initially, the **Capturing step** collects a calibrated set of RGB-D images and creates the original mesh (M_{original}). In the **Mesh Processing step**, this mesh is filtered using the room layout to produce a filtered mesh (M_{filtered}), which is then integrated with the layout to create the combined mesh (M_{combined}). A new set of RGB images is captured in the **Texture Image Acquisition step**, which is utilized in the second phase's **Texturing step**. Here, the M_{combined} and these RGB images are used to apply textures to the mesh. Following this, the **Plane2Image step** generates a set of images corresponding to the room's walls. These images are then refined in the **Post-Texturing step**, which finalizes the pipeline.

3.3 Capturing

The primary objective of the **Capturing step** is to scan a room, resulting in a collection of color and depth images along with their corresponding camera parameters (intrinsics and extrinsics). In the remainder of this paper, we will refer to these images as C_{original} and D_{original}, respectively. From the ensemble of camera intrinsic parameters (focal lengths f_x and f_y, and optical centers c_x and c_y) we derive K_{average}. K_{average} represents the mean intrinsic matrix, calculated by averaging the parameters of all cameras involved in the scan. To improve the capturing process, we develop a UI, depicted in Fig. 5a, which displays real-time visualizations of scanned and unscanned areas, as a toggle option. This feature helps in scanning all areas, ensuring complete meshes. Quality meshes are crucial, especially for occlusion tests to be accurately performed in the Texturing phase in Sec. 3.6, which directly affect texturing quality. We refer the reader to the supplementary (Section A.4) for more information about the occlusion tests.

Our UI relies on the creation of the room's mesh on-the-fly. The mesh formation algorithm is based on Newcombe's work, KinectFusion [28]. We exclude its pose tracking module, and instead we leverage $D_{original}$ and the camera parameters from Apple Inc. [4] for the depth fusion process. The resulting mesh is referred to as M_{original} and serves as input to the Mesh Processing step in Sec. 3.4.

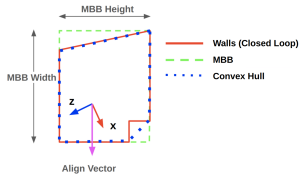
3.4 Mesh Processing

Our system categorizes rooms into open rooms and closed rooms. A closed room is defined by vertical walls forming a loop, enabling the creation of a watertight mesh fully enclosed by walls. In contrast, an open room lacks a closed loop of walls. For both room types, our pipeline initiates the **Mesh Processing step** by calculating a 2D Minimum Bounding Box (MBB) [2] in the X-Z coordinate plane, based on the previously computed room layout (see Fig. 3a for the coordinate system). The MBB represents the smallest 2D rectangular box that entirely encloses all vertical walls of a room. As illustrated in Fig. 3a, the outputs of the MBB calculation are $\text{MBB}_{\text{width}}$, $\text{MBB}_{\text{height}}$ and $\text{MBB}_{\text{align}}$. $\text{MBB}_{\text{width}}$ and $\text{MBB}_{\text{height}}$ define the spatial dimensions of the room. $\text{MBB}_{\text{align}}$, or Align Vector, is aligned with the $\text{MBB}_{\text{width}}$ edge, providing a reference for the subsequent processing stages in Sec. 3.7 and Sec. 3.8. Subsequently, the original mesh, M_{original}, is filtered using the room layout. The filtering step focuses on meshes excluding the room layout's mesh. Its goal is to eliminate meshes too close to the walls. The mesh must be filtered sufficiently for successful texturing of the layout's planar structures, while retaining enough information about object meshes for occlusion tests. This process, illustrated in Fig. 3b, is based on two criteria: (1) **Loop-Based**: This method eliminates mesh faces outside the 2D polygonal loop, defined by walls and the convex hull (see Fig. 3a) for closed rooms and open rooms respectively. (2) **Distance and Normal-Based**: Firstly, all mesh faces close to the walls are removed, shown as *Filtered by Distance* in Fig. 3b. Then, further away faces are removed based on their normals: faces are discarded if their normal vector forms an angle of less than a threshold with the wall's normal, depicted as *Filtered by Normal* in Fig. 3b. These thresholds used for mesh filtering are empirically determined and are reported in Sec. 4.1. The unfiltered region of M_{original}, is specified as *Intact Area* in Fig. 3b. The resulting filtered mesh, M_{filtered} is illustrated in Fig. 3d.

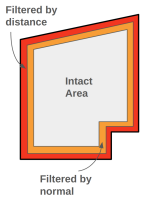
Finally, we perform the remeshing process, shown in Fig. 3c. Initially, every vertical wall is composed of two triangular faces. These walls are then remeshed by inserting grid points, using Constrained Delaunay Triangulation (CDT) [3]. CDT edges are inserted to encircle vertical planes. In the case of closed rooms, the floor and ceiling are formed by CDT by inserting constraining edges around the closed loop. Moreover, evenly distributed grid points and along-edge points are added within the loop. For open rooms, the convex hull shown in Fig. 3a is used to form the floor and ceiling, although it may not accurately represent the room's contour. The remeshed vertical and horizontal planes are integrated with M_{filtered} to form M_{combined}, shown in Fig. 3e. During this process, semantic information is precisely documented, linking each mesh segment to its respective semantic component. M_{combined} is the input to the texturing phase detailed in Sec. 3.6. More visualizations of the Mesh Processing step are shown in the supplementary (Section A.1).

3.5 Texture Images Acquisitions

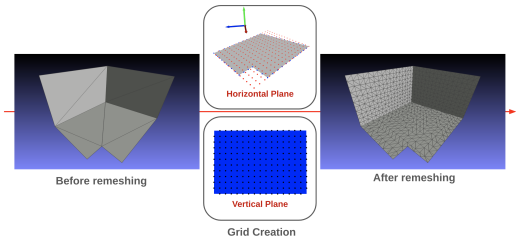
As outlined in Sec. 3.1, texturing algorithms [15, 37] encounter issues such as blurry textures (from using C_{original}) and visible texture



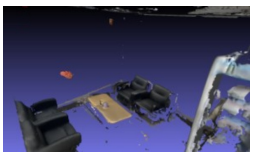
(a) Minimum bounding box (MBB) calculation.



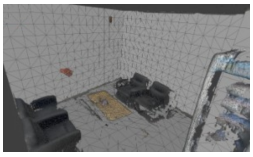
(b) Mesh filtering scheme.



(c) Remeshing scheme for structures of the room layout.



(d) Filtered mesh M_{filtered}.



(e) Combined mesh M_{combined}.

Figure 3: Processes and outputs of the Mesh Processing step.

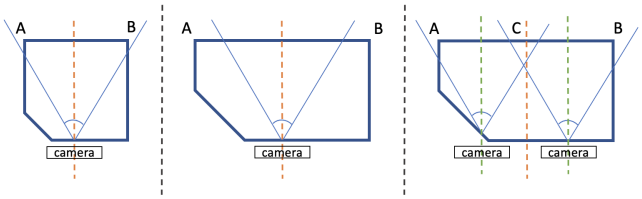
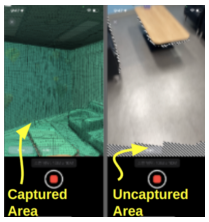


Figure 4: **Splitting scheme for large planes**: In the left image, a small plane is adequately covered by a single image. The middle image illustrates a large plane that extends beyond the scope of one image. In the right image, the large plane (AB) is divided into subplanes (AC and BC), each of which can be completely covered by a single image.

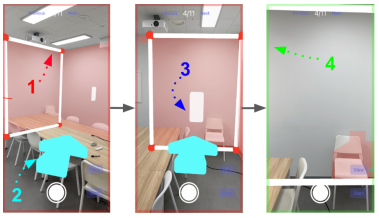
seams. To address these challenges, we introduce an AR-Guided texture image capturing approach specifically designed for texturing vertical walls. This method offers three key advantages. First, it allows users to capture images in a more stable manner, producing sharper images. Second, it seeks to minimize the number of images required per plane, as texturing with a single image reduces the misalignment issues associated with texturing using multiple images. Lastly, this strategy significantly speeds up the texturing process due to fewer images being used. When capturing large planes, multiple images may be needed due to limitations in camera field of view (FoV) or obstructions. To overcome this, we introduce the **Divide and Conquer** algorithm, which segments a large plane into smaller subplanes that can each be covered by a single image, whilst minimizing the overall number of images required.

3.5.1 Large Plane Dividing (Divide)

We develop a plane splitting algorithm that assesses the feasibility of capturing an entire plane in one shot as the user navigates the space (see Fig. 4). Using the phone's FoV in portrait mode, determined by K_{average}, we calculate the optimal distance d^* from which the camera should capture the plane of interest. From this, the optimal camera position, $\vec{X}^*$ is calculated as follows: $\vec{X}^* = \vec{c_p} + d^* \times \vec{n}_p$, where $\vec{c_p}$ and $\vec{n}_p$ are the plane of interest's center position and normal vector, respectively. With a standard ceiling height of 2.44 m [36], we propose that aligning the optimal camera height (in portrait mode capture) with the midpoint of the wall height (i.e. around 1.22 m) is ergonomically suitable for users. Then, we verify whether $\vec{X}^*$ remains within the room layout. If $\vec{X}^*$ is deemed too far or impractical, we recursively subdivide the plane into smaller subplanes, until the optimal conditions for capture are met. This recursion is guided by two critical criteria: the distance from the wall, crucial for capturing detailed and realistic images, and polygon inlier status, which checks if the camera position is within the room's polygonal boundary. All in all, our **divide** algorithm allows for capturing plane sections at closer and more effective distances. Refer to the supplementary material for more details (Section B, Algorithm 3).



(a) **UI for the Capturing Step**. The left view shows captured area and the right view shows uncaptured area.



(b) **UI for Divide and Conquer AR-Guided image capturing step**: guides are the target plane (1), optimal camera position indicators (2,3), and a green boundary (4) for proximity alerts.

Figure 5: UI for the **Capturing** and **Conquer step.**

3.5.2 Conquer

The **conquer** step aims to guide the user to the optimal $\vec{X}^*$ to capture images for texturing. For this, we introduce an AR guidance system that directs the user to $\vec{X}^*$. As shown in Fig. 5b, the target subplane, marked as (1), is emphasized by a red box with white edges. The user is directed towards the optimal capture position by a cyan arrow, marked as (2) in Fig. 5b. $\vec{X}^*$, marked as (3) in Fig. 5b, is visualized in AR as a white rectangular 3D phone model. The device interface displays a green boundary (marked as (4) in Fig. 5b) and provides haptic feedback to signal proximity to the target position. These guides ensure precise user alignment with the target $\vec{X}^*$. Upon capturing a target subplane, it reverts to its natural color, confirming success, whilst uncaptured subplanes are left in red. Images obtained through this process are designated as $C_{\text{recaptured}}$, marking the completion of the first phase of our pipeline.

3.6 Texturing

Starting with the second phase of our pipeline, the **Texturing** process aims to give realism to the combined mesh M_{combined} from Sec. 3.4. We utilize SOTA assignment-based texturing methods such as SeamlessTex [15] and MVSTex [37]. However, instead of using C_{original} from the Capturing stage (Sec. 3.3), our method employs $C_{\text{recaptured}}$ from Sec. 3.5, tailored for texturing. For further details on the texturing algorithms employed, please refer to the appendix (Section A.4) and relevant literature [15, 37, 45]. The resulting textured mesh, M_{textured}, includes both the textured room layout mesh (textured version of the remeshed room layout from Sec. 3.4) and the textured object mesh (textured version of M_{filtered} from Sec. 3.4). Utilizing semantic information retrieved during the Mesh Processing step, these components are systematically divided into distinct textured meshes for further processing.

3.7 Plane2Image

The aim of **Plane2Image** is to render 2D images from 3D textured planes as illustrated in Fig. 6. To do so, we use the semantically split textured meshes from Sec. 3.6 to ensure rendering without interference, and leverage the Metal [27] (for iOS) and OpenGL [43]

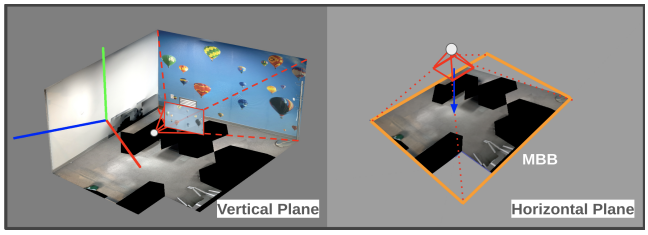


Figure 6: Plane2Image rendering operation for vertical and horizontal planes.

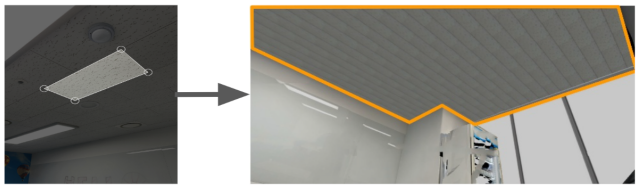


Figure 7: UI for capturing a ceiling sample (left), and a ceiling texture created using the captured sample (right).

(for Linux) frameworks. For vertical planes, we extract information of each plane such as the normal vector, center, width, and height from the database. A virtual camera is then positioned and oriented based on these parameters to precisely render the plane to a 2D image. For horizontal surfaces, we instead use the MBB parameters from Sec. 3.4. For more details on this rendering process, please consult the supplementary material (Section B, Algorithm 1 and Algorithm 2). This transition from 3D to 2D enables the application of various image processing techniques, such as image inpainting [8, 13, 25, 35] and generative AI methods [1, 29, 32, 47], as further outlined in Sec. 3.8. It is important to note that the remeshed vertical wall meshes from Sec. 3.4 are reverted back to their original two-face configuration, reducing the mesh's number of vertices and faces. This simplification aids in texture coordinate mapping. For vertical planes, vertices are assigned new texture coordinates based on only the four plane corners. Rendered images are then applied using these texture coordinates for the simplified faces. Horizontal surfaces undergo a similar face and vertex reduction, where the texture coordinates are mapped based on the MBB parameters.

3.8 Post Texturing

In Sec. 3.7, we established one-to-one correspondences between 3D planar structures and 2D images, simplifying the process of improving textures to merely switching images or altering them. In this section, we introduce various texturing enhancement methods that are performed by directly modifying information in the 2D image domain. Mainly, we propose two post texturing techniques, the **ImageMode** and the **SampleMode**, applicable to both vertical and horizontal surfaces.

The ImageMode allows users to replace the defective rendered 2D images from the Plane2Image phase with any image of their choice, offering high versatility and customizability. Specifically, image inpainting models [8, 13, 25, 35] successfully address both the environmental and technological challenges by filling in untextured areas (from occlusions), correcting texture artifacts, and removing undesired contents. This significantly enhances the overall quality and accuracy of the texturing outputs, as shown in Fig. 8 for the ZIT inpainting model [13] used by RoomRecon. ZIT inpainting is chosen as it provides the best trade-off between quality and inference speed relative to other methods [25, 32, 35]. For a detailed rationale behind this choice, please refer to the supplementary material (Section A.5). On top of inpainting, stable diffusion-based methods [32, 47] allow for the addition of creative content to original textures, allowing customization of room decorations according to user preferences. For additional visualizations, please refer to the supplementary material (Section A.5). The SampleMode targets the challenges of achieving high-quality textures for floors and ceilings. Users have the option to select a sample from a database or capture one from real life. Therefore, we provide a UI for capturing samples from these surfaces as shown in Fig. 7. Users can customize their floor and ceiling textures by adjusting parameters such as sample width, sample height, sample angle and sample offset, allowing for the creation of textures that closely resemble the real world. An example of a ceiling texture, created using our SampleMode, is shown in Fig. 7. For further visualization of textures created using SampleMode, refer to the supplementary material (Section A.2). This marks the end of the RoomRecon pipeline.

4 Experiments

In this section, we assess the performance of various texturing methods within our RoomRecon pipeline, with a focus on both local and global texture quality, and time complexity. For a complete and comprehensive result of our experiments, please consult the supplementary materials (Section C).

4.1 Experiment Setup

In our paper, we conduct experiments within diverse indoor spaces, including seven rooms of office environments and six rooms of household environments. The data acquisition, involving RGB-D images and camera parameters, is performed using the iPhone 12 Pro, integrated with the ARKit framework [4]. Room layouts are obtained through Apple Inc.'s API [5]. Detailed specifications for each room are available in the supplementary material (Section C). We compare the texture capabilities of six texturing methods, reimplemented within our RoomRecon texturing pipeline: ColorMapOpt [46], PlaneOpt [38], MVSTex [37], SeamlessTex [15], and our adaptations, Our-MVSTex and Our-SeamlessTex. We refer to our adaptations as *Ours* from now onwards. SeamlessTex [15] originally includes camera pose optimization and texture synthesis processes, which we omit in our adaptation due to their high computational demands. *Ours* utilizes newly captured images $C_{\text{recaptured}}$ for texturing, whereas standard algorithms such as MVSTex, SeamlessTex, ColorMapOpt, and PlaneOpt use sampled images C_{sampled}, selected from C_{original} every four timesteps based on sharpness. Typically, the other three images are discarded unless their poses significantly differ from the selected image. All other aspects of the RoomRecon pipeline remain identical across all experiments.

In the Capturing phase, we set the voxel size to 20 mm and utilize 192×256 RGB-D images, leveraging camera parameters provided by ARKit [4]. Real-time mesh generation is facilitated through the Metal framework [27]. The initial mesh (M_{original}) undergoes a 50% reduction in face count using Garland's simplification algorithm [18]. During the mesh filtering scheme in Sec. 3.4, faces situated within $0-5$ cm from a plane are excluded via the *filtered by distance* approach, while those within $5-10$ cm are filtered using the *filtered by normal* strategy, with a threshold angle of 10 degrees. In the remeshing stage of Sec. 3.4, we specify a grid resolution of five points per meter for vertical planes and a uniform grid of 15×15 points throughout the MBB for horizontal planes. For the Plane2Image stage, rendering parameters are configured such that each meter of the plane corresponds to 500 pixels in the resulting image. As an illustration, a plane measuring 2 m in width and 1 m in height is rendered as an image with dimensions 1000×500 pixels. Local Texture Quality Evaluation (Sec. 4.2) and Global Plane-to-Plane Evaluation (Sec. 4.3) experiments are conducted prior to the application of inpainting or any Post Texturing processes. For both evaluations, the focus is exclusively on textures of vertical walls, excluding floors, ceilings, and object textures. This is because

Table 1: Results of local and global plane-to-plane texture quality assessments (standard deviations in parentheses).

	Local Texture Quality Eval.				Global Plane-to-Plane Texture Quality Eval.			
Method	PSNR[dB]↑	SSIM[-]↑	ST-LPIPS[-]↓	Blurriness[-]↓	PSNR[dB]↑	SSIM[-]↑	ST-LPIPS[-]↓	Blurriness[-]↓
ColorMapOpt [46]	20.19 (±1.48)	0.84 (±0.02)	0.17 (±0.02)	0.25 (±0.04)	16.09 (±1.64)	0.77 (±0.06)	0.25 (±0.06)	0.44 (±0.05)
PlaneOpt [38]	20.13 (±0.99)	0.84 (±0.02)	0.17 (±0.02)	0.22 (±0.04)	16.49 (±1.72)	0.77 (±0.06)	0.25 (±0.06)	0.41 (±0.04)
MVSTex [37]	20.87 (±1.79)	**0.86** (±0.03)	0.10 (±0.01)	0.22 (±0.03)	17.09 (±2.12)	0.80 (±0.06)	0.19 (±0.04)	0.33 (±0.02)
SeamlessTex [15]	21.32 (±1.61)	**0.87** (±0.03)	0.09 (±0.02)	0.23 (±0.03)	17.61 (±2.18)	0.81 (±0.06)	0.18 (±0.04)	0.34 (±0.03)
Our-MVSTex	**21.35** (±1.40)	0.86 (±0.03)	**0.09** (±0.02)	**0.20** (±0.02)	**20.35** (±1.94)	**0.91** (±0.02)	**0.11** (±0.02)	**0.29** (±0.02)
Our-SeamlessTex	**21.33** (±1.40)	0.86 (±0.03)	**0.09** (±0.02)	**0.20** (±0.02)	**20.80** (±2.27)	**0.92** (±0.02)	**0.10** (±0.02)	**0.29** (±0.02)

Table 2: Blurriness scores of original, sampled and recaptured images (left). Processing time of different texturing methods (right).

	Orig.	Sampled	Recapt.
Num. of Frames[-]	599.92 (± 262.90)	258.85 (± 106.98)	11.38 (± 3.82)
Blur [-]	0.41 (± 0.06)	0.38 (± 0.05)	0.25 (± 0.02)

Methods	Time[s]
ColorMapOpt	78.50(± 35.48)
PlaneOpt	273.79(± 179.96)
MVSTex	8.48(± 2.03)
SeamlessTex	32.62(± 18.93)
Our-MVSTex	3.23(± 1.05)
Our-SeamlessTex	4.44(± 1.29)

floors and ceilings are regenerated using the SampleMode in the Post Texturing step (Sec. 3.8), and our focus is on texturing structural elements, not objects.

4.2 Local Texture Quality Evaluation

Methodology: The objective is to assess local texture quality of walls, with the following evaluation procedure. Ground truth (GT) images and corresponding camera parameters are captured using the RoomRecon framework, focusing on semantically important regions within the room that have diverse colors, lines and edges (e.g.: wall paintings, doors and other highly detailed areas). We avoid uniform regions such as plain white walls, where texturing errors are hard to spot. For samples of these captured ground truth images, refer to the supplementary material (Sec. A.6). The camera intrinsics and extrinsics are tracked by ARKit [4]. Each texturing method is applied to the room mesh, and the resulting textured meshes are rendered at the corresponding GT poses (i.e. poses at which the GT images were captured) for comparison in the 2D image space. This evaluation is referred to as *local texture quality assessment* because it examines only the specific, limited areas of the scene captured by the purposely curated GT. Consequently, the evaluation focuses on partial areas, whether it be portions of a single plane or encompassing multiple planes. Masked GT images and masked rendered images are compared for four metrics, namely PSNR [42], SSIM [39], Shift Tolerant Learned Perceptual Image Patch Similarity (ST-LPIPS) [19], and Blurriness [11]. These metrics are selected, as done by prior work [15, 45], to ascertain the similarity of textured rendered outputs to GT images (PSNR, SSIM, and ST-LPIPS), and to evaluate the sharpness and vividness of the textured outputs using the Blurriness measure, which is GT-independent. Specifically, PSNR compares pixel values and ranges from $0-60$ dB, whilst SSIM compares image properties (e.g.: luminance, contrast and structure) on a scale from $0-1$.

Discussion: The left section of Tab. 1 highlights the performance of six texturing methods on local texture quality. Our methods outperform all previous methods across most metrics except in SSIM and ST-LPIPS, where the scores are comparable to those of the baseline methods: SeamlessTex [15] and MVSTex [37]. This similarity in SSIM and in ST-LPIPS can be attributed to the fact that SeamlessTex and MVSTex try to use the same image to texture groups of neighboring mesh faces. Hence, in the context of our local texture quality assessment, it is highly likely that all meshes are textured with the same image, leading to few seams and structural discrepancies. In this setting, our methods have small margins for improvement, as we also try to minimize the number of images used for texturing.

On the other hand, our techniques demonstrate a significant advantage in PSNR and Blurriness. As shown in Tab. 2, the original set of color images C_{original} exhibits a high Blurriness score of 0.41, and even the sharpest sampled C_{sampled}, used by the prior methods, only improve the Blurriness to 0.38. In contrast, the $C_{\text{recaptured}}$ through our Divide-And-Conquer scheme are 39% and 34% sharper than the original and sampled images, respectively. This leads to a 9% and 13% sharper texture output compared to MVSTex and SeamlessTex respectively, as our $C_{\text{recaptured}}$ images are utilized for the texturing process. Additionally, our superior performance in PSNR demonstrates that our methods more closely resemble the GT images in terms of pixel values. Finally, we notice that our methods perform similarly across all metrics, irrespective of the underlying texturing method. This shows that using few, high-quality input images to the texturing module is of upmost importance, reducing the need for complicated texturing algorithm.

4.3 Global Plane-to-Plane Texture Evaluation

Methodology: This study compares entire plane textures against high-quality GT images. GT images of full walls are captured with mobile phones, employing techniques such as 0.6x zoom and panorama shots. These images undergo a homography transform, where the four vertices of the walls are manually clicked to create GT planar images for the respective walls. Then, the images are refined using Adobe Photoshop [1] to establish high-quality benchmarks for comparison, including the removal of furniture from images. During the evaluation, the rendered images of textured planes from the Plane2Image stage are compared to these GT images. The evaluation utilizes the same four metrics employed in Sec. 4.2.

Discussion: The right section of Tab. 1 presents the performance of six texturing methods within a global plane-to-plane evaluation framework. *Ours* demonstrate significant improvements across all metrics compared to prior work. For example, for Our-SeamlessTex, PSNR improves by 18.1%, SSIM by 13.6%, ST-LPIPS by 44.4%, and Blurriness by 14.71%, compared to its baseline SeamlessTex. The Blurriness reductions and PSNR improvements align with findings from the local evaluation in Sec. 4.2, highlighting the effect of using sharper images. Moreover, unlike in the local results, our methods greatly surpass their baselines in SSIM and ST-LPIPS. This demonstrates that using fewer, carefully captured images through our divide-and-conquer scheme, leads to rendered images that more closely replicate the GT image properties in terms of structure and color. Moreover, as each image captures a significant portion of a wall, it ensures that all structures and details of that wall area are intact and free of texture seams. Again, the similar scores between our methods suggests that using quality input data for texturing removes the need for highly complex texturing algorithms. For more qualitative comparisons of textured results using PlaneOpt, SeamlessTex, and Our-SeamlessTex, along with ZIT-inpainted [13] version of Our-SeamlessTex, refer to Fig. 8. This figure illustrates the visual enhancements and the detailed realism achieved through our texturing methods, providing a clearer understanding of the quantitative results discussed above. Full rooms created following our RoomRecon pipeline with Our-SeamlessTex are visualized in Fig. 9.

4.4 Time Analysis of Texturing Methods and RoomRecon Pipeline

We examine the texturing times of various methods on a Linux machine with AMD Ryzen 9 5900x 12-core × 24 processor and report our findings in the right part of Tab. 2. Our approach considerably accelerates the texturing process, achieving times of just 3.23 s and 4.44 s for our-MVSTex and our-SeamlessTex respectively. This is a notable increase in speed compared to their baselines that run in 8.48 s and 32.62 s respectively. Our method achieves increased efficiency primarily through the implementation of our AR-Guided image capturing approach. This method minimizes the number of captures required to cover a room, resulting in an average of 11.38 images per room, a significant reduction from the typical 258.85 images needed (C_{sampled}), as stated in the left side of Tab. 2. Using fewer images for texturing simplifies the selection of optimal images for each face, leading to shorter texturing time. Additionally, we provide a breakdown of the time each step takes in our RoomRecon pipeline using the standard MVSTex [37] and Our-MVSTex on an iPhone 12 Max, as outlined in Tab. 3. The pipeline steps common to both texturing methods include the room layout parsing, Mesh Processing (Sec. 3.4), Plane2Image (Sec. 3.7), and the display of textured meshes (M_{textured}) on the phone's screen. These combined common processes take an average of 4.8 s to compute. As RoomRecon's texturing module differs from prior works, a time analysis of MVSTex [37] and our-MVSTex is also outlined in Tab. 3. In the implementation of MVSTex [37], the texturing step involves calculating the Blurriness score for each image in C_{original} to sample only the sharpest images for texturing (C_{sampled}). This step and the actual MVSTex take an average of 4.85 s and 18.71 s respectively. With our new Divide and Conquer scheme, we eliminate the need to calculate blurriness, as we recapture few sharp images for texturing. The Divide stage is practically instantaneous (0.23 ms), and the significantly fewer images used for our texturing (from 258.85 to 11.38) lead to a total texturing time of just 4.72 s. Thus, RoomRecon with our-MVSTex is processed in 9.48 s, whereas RoomRecon with MVSTex runs three times slower. Our simplified approach reduces waiting time for users as their models are processed more rapidly. However, it's important to acknowledge that our method incorporates a second capturing step, not factored into the time analysis provided, which solely focused on algorithm processing time. This additional image acquisition stage introduces a time overhead, dependent upon the user's familiarity with the RoomRecon application. Therefore, the overall time required for an experienced user to generate a virtual textured room remains comparable irrespective of the method used, yet our approach places less strain on the phone.

4.5 User Evaluation

We conduct a user study to assess how different people judge the quality of different texturing methods. After filling in a consent form, a survey is given to each participant, with the task of selecting the *best* rendered image amongst a set of rendered images textured by different methods. We explain to participants to judge based on uniform and consistent illumination, texture seams and clarity. Specifically, we randomly select 20 plane images, each textured with these four texturing methods: MVSTex [37], SeamlessTex [15], our-MVSTex, and our-SeamlessTex, such that a total of 80 samples are provided. A group of four images for each scene are randomly displayed. ZIT inpainting [13] is utilized to fill untextured areas in all rendered images. For each scene, we show the four differently textured images side by side and ask the participants to record their preferences.

A group of 30 people with different backgrounds are asked to complete the survey. The age and background distribution are as follows: 14 software engineers, 6 designers, and 10 computer science students. There are 18 males (average age 33.3 ± 5.0) and 12 females (average age 30.2 ± 4.1), with an overall average age of 32.1 ± 4.8 years. In total, 600 selections are recorded (1 per person per scene), and the voting results for each texturing method are summarized in Tab. 4. Most participants chose our-MVSTex and our-SeamlessTex as the best methods, with voting rates of 43.6% and 42.1% respectively. This shows the qualitative superiority of our textured meshes, emphasizing the positive effect of our proposed techniques on texture quality, clarity and seam reduction.

Table 3: Detailed process and texturing time analysis on iPhone for MVSTex [37] and Our-MVSTex.

Common Process (a)		
Layout Parsing [s]	(3.70 ± 1.30)e-3	
Mesh Proc. [s]	0.23 ± 0.17	
Plane2Img [s]	3.93 ± 1.46	
Display Results [s]	0.60 ± 0.35	
Texturing Process (b)	**Original MVSTex.**	**Divide and Conquer (Ours)**
Frames [-]	599.92 ± 262.90	11.38 ± 3.82
Blur Calc. [s]	4.85 ± 2.24	–
Divide Calc. [s]	–	(0.23 ± 0.04)e-3
Texturing Time [s]	18.71 ± 9.25	4.72 ± 2.06
RoomRecon Total (a + b) [s]	28.33 ± 12.77	9.48 ± 3.69

Table 4: User study votes for the best texture solution.

Method	MVSTex	SeamlessTex	our-MVSTex	our-SeamlessTex
Number of votes	45(7.5%)	35(6.8%)	262(43.6%)	258(42.1%)

5 Limitations and Future Work

Whilst RoomRecon successfully demonstrates better texturing quality compared to previous methods, it presents some limitations. Firstly, our divide-and-conquer capturing method is restricted in narrow corridors as it cannot find suitable capture positions inside the room. Hence, it is essential to further refine the algorithm to handle such edge cases. Next, the current prototype does not optimize for user interaction time, which is important for the overall user experience. Lastly, the absence of a thorough user evaluation limits the understanding of the proposed UI's usability across different user demographics. The last two limitations come from the fact that this paper focuses on presenting a pipeline to improve texturing quality. We suggest a UI configuration to prove that involving users can improve texturing. However, our UI represents one potential approach, and we believe that the design and evaluation of an effective UI considering speed and ease of use can be the target of future work. Aside from tackling the above points, future work can focus on expanding RoomRecon's capabilities to multi-room settings, allowing for the capture of entire floors (see supplementary material, Section A.3). Moreover, integrating practical requirements from experts in target fields such as real estate and interior design would improve RoomRecon's performance and 3D modeling capabilities in these target applications.

6 Conclusion

In this study, we introduced RoomRecon, a novel mobile application designed for the precise and realistic texturing of indoor spaces. By leveraging the capabilities of mobile devices and focusing on the static elements of rooms, RoomRecon allows users to generate detailed 3D models with minimal need for updates. This application tackles limitations of current 3D reconstruction technologies by offering an AR-guided UI that supports the capturing experience. Through a dedicated two-phase pipeline, RoomRecon addresses common texturing issues such as blurring and seams, outperforming traditional methods both quantitatively and qualitatively.

Figure 8: Qualitative results of textured planes using PlaneOpt [38], SeamlessTex [15], Our-SeamlessTex, and Our-SeamlessTex after inpainting operation. For more visualizations of results using Ours-Inpainted, refer to the supplementary material (Section A.7).

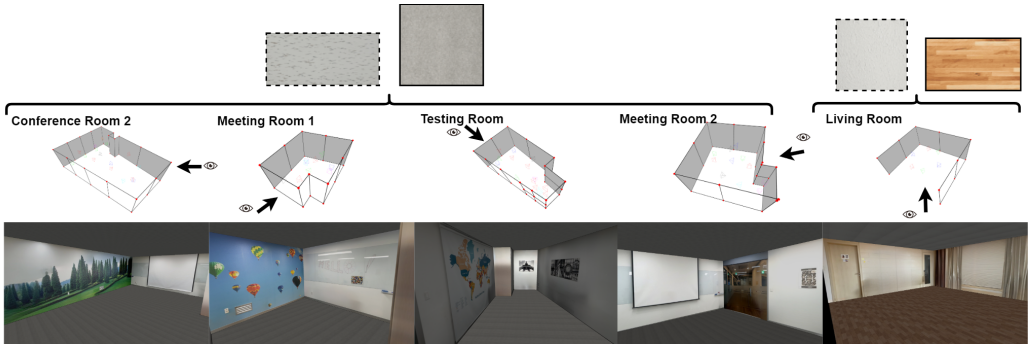


Figure 9: Textured room layouts after completing our RoomRecon pipeline. Samples with dashed lines show ceiling samples used for creating ceiling textures and samples in solid lines show floor samples for floor textures. For additional viewpoints of the textured layouts, refer to the supplementary material (Section A.7).

References

[1] Adobe. Official Adobe Photoshop - Photo & Design Software — adobe.com. https://www.adobe.com/products/photoshop.html. [Accessed 07-05-2024]. 2, 5, 6

[2] A. Andrew. Another efficient algorithm for convex hulls in two dimensions. *Information Processing Letters*, 9(5):216–219, dec 1979. doi: 10.1016/0020-0190(79)90072-3 3, 11, 21

[3] M. V. Anglada. An improved incremental algorithm for constructing restricted Delaunay triangulations. *Computers & Graphics*, 21(2):215–223, mar 1997. Graphics Hardware. doi: 10.1016/S0097-8493(96)00085-4 3, 21

[4] Apple Inc. ARKit 6 - Augmented Reality - Apple Developer — developer.apple.com. https://developer.apple.com/augmented-reality/arkit/. [Accessed 07-05-2024]. 1, 3, 5, 6, 12

[5] Apple Inc. RoomPlan - Augmented Reality - Apple Developer — developer.apple.com. https://developer.apple.com/augmented-reality/roomplan/. [Accessed 07-05-2024]. 2, 5

[6] A. Avetisyan, T. Khanova, C. Choy, D. Dash, A. Dai, and M. Nießner. SceneCad: Predicting Object Alignments and Layouts in RGB-D Scans. In *Proc. ECCV*. Springer Nature, Berlin, Germany, August 2020. 2

[7] S. Bi, N. K. Kalantari, and R. Ramamoorthi. Patch-based optimization for image-based texture mapping. *ACM Trans. Graph.*, 36(4), jul 2017. doi: 10.1145/3072959.3073610 1

[8] C. Cao, Q. Dong, and Y. Fu. Zits++: Image inpainting by improving the incremental transformer on structural priors. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(10):12667–12684, oct 2023. doi: 10.1109/TPAMI.2023.3280222 5, 13

[9] X. Chen, H. Zhao, G. Zhou, and Y.-Q. Zhang. PQ-Transformer: Jointly Parsing 3D Objects and Layouts From Point Clouds. *IEEE Robotics and Automation Letters*, 7(2):2519–2526, apr 2022. doi: 10.1109/LRA.2022.3143224 2

[10] P. Cipresso, I. A. C. Giglioli, M. A. Raya, and G. Riva. The Past, Present, and Future of Virtual and Augmented Reality Research: A Network and Cluster Analysis of the Literature. *Frontiers in Psychology*, 9, nov 2018. doi: 10.3389/fpsyg.2018.02086 1

[11] F. Crete, T. Dolmiere, P. Ladret, and M. Nicolas. The blur effect: perception and estimation with a new no-reference perceptual blur metric. In B. E. Rogowitz, T. N. Pappas, and S. J. Daly, eds., *Human Vision and Electronic Imaging XII*, vol. 6492, p. 64920I. International Society for Optics and Photonics, SPIE, California, USA, mar 2007. doi: 10.1117/12.702790 6, 22

[12] A. Dai, M. Nießner, M. Zollhöfer, S. Izadi, and C. Theobalt. BundleFusion: Real-Time Globally Consistent 3D Reconstruction Using On-the-Fly Surface Reintegration. *ACM Transactions on Graphics (TOG)*, 36(3), may 2017. doi: 10.1145/3054739 1

[13] Q. Dong, C. Cao, and Y. Fu. Incremental transformer structure enhanced image inpainting with masking positional encoding. In *Proc. CVPR*, pp. 11348–11358. IEEE Computer Society, Los Alamitos, CA, USA, jun 2022. doi: 10.1109/CVPR52688.2022.01107 2, 5, 6, 7, 13, 14

[14] H. Fu, R. Jia, L. Gao, M. Gong, B. Zhao, S. Maybank, and D. Tao. 3D-FUTURE: 3D Furniture Shape with TextURE. *International Journal of Computer Vision*, 129(12):3313–3337, dec 2021. doi: 10.1007/s11263-021-01534-z 2

[15] Y. Fu, Q. Yan, J. Liao, H. Zhou, J. Tang, and C. Xiao. Seamless texture optimization for rgb-d reconstruction. *IEEE Transactions on Visualization & Computer Graphics*, 29(03):1845–1859, mar 2023. doi: 10.1109/TVCG.2021.3134105 1, 2, 3, 4, 5, 6, 7, 8, 12, 22, 23

[16] Y. Fu, Q. Yan, L. Yang, J. Liao, and C. Xiao. Texture Mapping for 3D Reconstruction with RGB-D Sensor. In *Proc. CVPR*, pp. 4645–4653. IEEE Computer Society, Los Alamitos, CA, USA, jun 2018. doi: 10.1109/CVPR.2018.00488 2

[17] H.-a. Gao, B. Tian, P. Li, X. Chen, H. Zhao, G. Zhou, Y. Chen, and H. Zha. From Semi-supervised to Omni-supervised Room Layout Estimation Using Point Clouds. In *Proc. ICRA*, pp. 2803–2810. Institute of Electrical and Electronics Engineers (IEEE), New Jersey, United States, 2023. doi: 10.1109/ICRA48891.2023.10161273 2

[18] M. Garland and P. S. Heckbert. Surface simplification using quadric error metrics. In *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '97, p. 209–216. ACM Press/Addison-Wesley Publishing Co., New York, USA, 1997. doi: 10.1145/258734.258849 5

[19] A. Ghildyal and F. Liu. Shift-Tolerant Perceptual Similarity Metric. In *Proc. ECCV*, pp. 91–107. Springer Nature, Cham, Switzerland, 2022. 6

[20] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial networks. *Commun. ACM*, 63(11):139–144, oct 2020. doi: 10.1145/3422622 2

[21] Google Inc. Build new augmented reality experiences that seamlessly blend the digital and physical worlds — ARCore — Google for Developers — developers.google.com. https://developers.google.com/ar. [Accessed 07-05-2024]. 1

[22] L. Höllein, A. Cao, A. Owens, J. Johnson, and M. Nießner. Text2Room: Extracting Textured 3D Meshes from 2D Text-to-Image Models. In *Proc. ICCV*, pp. 7909–7920. Institute of Electrical and Electronics Engineers (IEEE), New Jersey, United States, October 2023. 1, 2

[23] J. Huang, J. Thies, A. Dai, A. Kundu, C. Jiang, L. J. Guibas, M. Niessner, and T. Funkhouser. Adversarial Texture Optimization From RGB-D Scans. In *Proc. CVPR*, pp. 1556–1565. IEEE Computer Society, Los Alamitos, CA, USA, jun 2020. doi: 10.1109/CVPR42600.2020.00163 2

[24] E. Karaca and M. A. Tunga. Interpolation-based image inpainting in color images using high dimensional model representation. In *2016 24th European Signal Processing Conference (EUSIPCO)*, pp. 2425–2429, 2016. doi: 10.1109/EUSIPCO.2016.7760684 2

[25] W. Li, Z. Lin, K. Zhou, L. Qi, Y. Wang, and J. Jia. Mat: Mask-aware transformer for large hole image inpainting. In *Proc. CVPR*, pp. 10748–10758. IEEE Computer Society, Los Alamitos, CA, USA, jun 2022. doi: 10.1109/CVPR52688.2022.01049 2, 5, 14

[26] K. Maninis, S. Popov, M. Niesner, and V. Ferrari. Vid2CAD: CAD Model Alignment Using Multi-View Constraints From Videos. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(01):1320–1327, jan 2023. doi: 10.1109/TPAMI.2022.3146082 2

[27] Metal. Metal Overview - Apple Developer — developer.apple.com. https://developer.apple.com/metal/. [Accessed 07-05-2024]. 4, 5

[28] R. A. Newcombe, A. Fitzgibbon, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohi, J. Shotton, and S. Hodges. KinectFusion: Real-time dense surface mapping and tracking. In *Proc. ISMAR*, pp. 127–136. IEEE Computer Society, Los Alamitos, CA, USA, oct 2011. doi: 10.1109/ISMAR.2011.6092378 1, 3

[29] OpenAI. Dall·e 3. https://openai.com/index/dall-e-3. [Accessed 07-05-2024]. 2, 5

[30] M. Portman, A. Natapov, and D. Fisher-Gewirtzman. To go where no man has gone before: Virtual reality in architecture, landscape architecture and environmental planning. *Computers, Environment and Urban Systems*, 54:376–384, 06 2015. doi: 10.1016/j.compenvurbsys.2015.05.001 1

[31] Z. Qin, Q. Zeng, Y. Zong, and F. Xu. Image inpainting based on deep learning: A review. *Displays*, 69:102028, 2021. doi: 10.1016/j.displa.2021.102028 2

[32] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proc. CVPR*, pp. 10674–10685. IEEE Computer Society, Los Alamitos, CA, USA, jun 2022. doi: 10.1109/CVPR52688.2022.01042 2, 5

[33] J. Schult, S. Tsai, L. Höllein, B. Wu, J. Wang, C.-Y. Ma, K. Li, X. Wang, F. Wimbauer, Z. He, P. Zhang, B. Leibe, P. Vajda, and J. Hou. ControlRoom3D: Room Generation using Semantic Proxy Rooms. In *Proc. CVPR*. Institute of Electrical and Electronics Engineers (IEEE), New Jersey, United States, 2024. 1, 2

[34] L. Song, L. Cao, H. Xu, K. Kang, F. Tang, J. Yuan, and Z. Yang. RoomDreamer: Text-Driven 3D Indoor Scene Synthesis with Coherent Geometry and Texture. In *Proceedings of the 31st ACM International Conference on Multimedia*, MM '23, p. 6898–6906. Association for Computing Machinery, New York, NY, USA, 2023. doi: 10.

1145/3581783.3611800 1, 2

[35] R. Suvorov, E. Logacheva, A. Mashikhin, A. Remizova, A. Ashukha, A. Silvestrov, N. Kong, H. Goka, K. Park, and V. Lempitsky. Resolution-robust large mask inpainting with fourier convolutions. In *Proc. WACV*, pp. 3172–3182. IEEE Computer Society, Los Alamitos, CA, USA, jan 2022. doi: 10.1109/WACV51458.2022.00323 2, 5, 14

[36] O. Vartanian, G. Navarrete, A. Chatterjee, L. B. Fich, J. L. Gonzalez-Mora, H. Leder, C. Modroño, M. Nadal, N. Rostrup, and M. Skov. Architectural design and the brain: Effects of ceiling height and perceived enclosure on beauty judgments and approach-avoidance decisions. *Journal of Environmental Psychology*, 41:10–18, 2015. doi: 10.1016/j.jenvp.2014.11.006 4

[37] M. Waechter, N. Moehrle, and M. Goesele. Let There Be Color! Large-Scale Texturing of 3D Reconstructions. In *Proc. ECCV*, pp. 836–850. Springer International Publishing, Berlin, Germany, 2014. 1, 2, 3, 4, 5, 6, 7, 12, 21, 22, 23

[38] C. Wang and X. Guo. Plane-Based Optimization of Geometry and Texture for RGB-D Reconstruction of Indoor Scenes. In *Proc. 3DV*, pp. 533–541. IEEE Computer Society, Los Alamitos, CA, USA, sep 2018. doi: 10.1109/3DV.2018.00067 2, 5, 6, 8, 22, 23

[39] Z. Wang, A. Bovik, H. Sheikh, and E. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004. doi: 10.1109/TIP.2003.819861 6

[40] M. Weerasinghe, K. Čopič Pucihar, J. Ducasse, A. Quigley, A. Toniolo, A. Miguel, N. Caluya, and M. Kljun. Exploring the future building: representational effects on projecting oneself into the future office space. *Virtual Reality*, 27(1):51–70, Mar 2023. doi: 10.1007/s10055-022-00673-z 1

[41] T. Whelan, R. F. Salas-Moreno, B. Glocker, A. J. Davison, and S. Leutenegger. ElasticFusion: Real-time dense SLAM and light source estimation. *The International Journal of Robotics Research*, 35(14):1697–1716, sep 2016. doi: 10.1177/0278364916669237 1

[42] S. Winkler and P. Mohandas. The evolution of video quality measurement: From psnr to hybrid metrics. *IEEE Transactions on Broadcasting*, 54:660 – 668, oct 2008. doi: 10.1109/TBC.2008.2000733 6

[43] M. Woo, J. Neider, T. Davis, and D. Shreiner. *OpenGL programming guide: the official guide to learning OpenGL, version 1.2*. Addison-Wesley Longman Publishing Co., Inc., 1999. 4

[44] X. Yang, L. Zhou, H. Jiang, Z. Tang, Y. Wang, H. Bao, and G. Zhang. Mobile3DRecon: Real-time Monocular 3D Reconstruction on a Mobile Phone. *IEEE Transactions on Visualization and Computer Graphics*, 26(12):3446–3456, dec 2020. doi: 10.1109/TVCG.2020.3023634 1

[45] X. Zhao, Z. Zhao, and A. G. Schwing. Initialization and Alignment for Adversarial Texture Optimization. In *Proc. ECCV*. Springer, Berlin, Germany, 2022. 2, 4, 6

[46] Q.-Y. Zhou and V. Koltun. Color map optimization for 3D reconstruction with consumer depth cameras. *ACM Trans. Graph.*, 33(4), jul 2014. doi: 10.1145/2601097.2601134 2, 5, 6, 22, 23

[47] J. Zhuang, Y. Zeng, W. Liu, C. Yuan, and K. Chen. A task is worth one word: Learning with task prompts for high-quality versatile image inpainting. *ArXiv*, abs/2312.03594, 2023. 2, 5

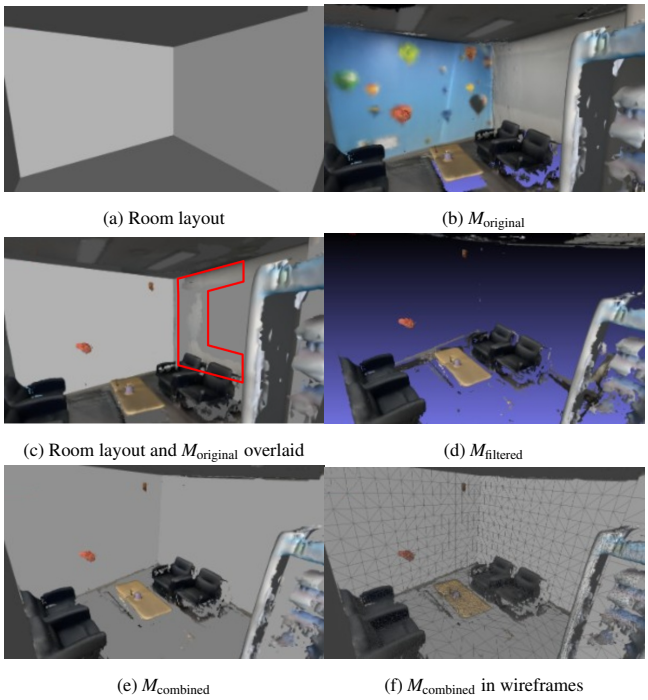
[48] J. Zhuang, Y. Zeng, W. Liu, C. Yuan, and K. Chen. A task is worth one word: Learning with task prompts for high-quality versatile image inpainting. *ArXiv*, abs/2312.03594, 2023. 13

Overview of Supplementary Material

1. Section A: We provide further visualizations of the RoomRecon processes and results.
2. Section B: We provide pseudocodes that explain the related algorithms in the main paper.
3. Section C: We present extensive tables of experiments showing quantitative statistics on the test datasets.

7 Further Visualizations of the RoomRecon Processes and Results

7.1 Detailed visualizations of the Mesh Processing step



(a) Room layout (b) M_{original}

(c) Room layout and M_{original} overlaid (d) M_{filtered}

(e) M_{combined} (f) M_{combined} in wireframes

Figure 10: Detailed views of the Mesh Processing step, showing various stages from the initial room layout and M_{original} to the final processed mesh representation M_{combined}.

Fig. 10 provides detailed insights into the Mesh Processing step (Section 3.4). Starting with the room layout (Fig. 10a) and the original mesh (Fig. 10b), which is derived from the Capturing step (Section 3.3) as M_{original}, the process continues with the mesh filtering process which eliminates mesh faces near the walls of the room layout structures (Fig. 10c), resulting in the filtered mesh, M_{filtered} (Fig. 10d).
After the remeshing procedures are applied to the structural elements (vertical and horizontal walls), these remeshed structural meshes are merged with the filtered mesh. The result is the creation of the combined mesh (Fig. 10e and Fig. 10f), denoted as M_{combined}, which serves as input for the subsequent Texturing step (Section 3.6).

7.2 Further visualizations of the SampleMode of the Post Texturing step

In the Post Texturing step (Section 3.8) of RoomRecon, texturing of walls can be done using the SampleMode. This approach allows the user to select either a custom or captured sample, which is then iteratively replicated to cover a specified plane of interest. Four adjustable parameters are provided as shown in Fig. 11:

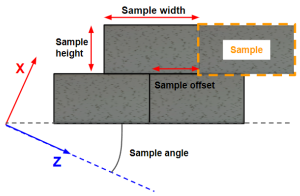
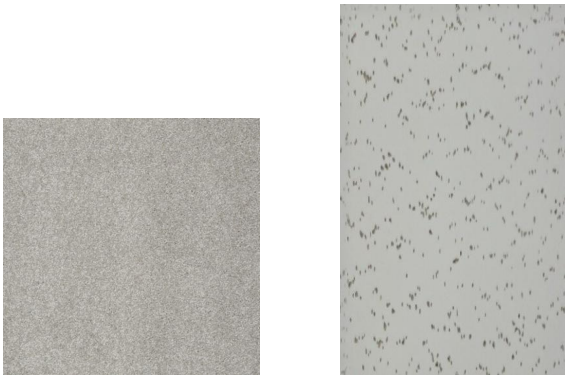


Figure 11: SampleMode parameters for the Post Texturing step.

1. Sample Width and Sample Height: Specify the dimensions of each sample.
2. Sample Offset: Specifies the spacing between consecutive samples.
3. Sample Angle: Allows rotation of the sample relative to the MBB_{align}, the vector derived from the Mesh Processing step (Section 3.4) during the Minimum Bounding Box (MBB) [2] calculation.

Using samples shown in Fig. 12a and Fig. 12b, floor and ceiling textures can be created using the SampleMode as shown in Fig. 13 and Fig. 14, respectively.



(a) Floor sample used to generate floor texture (b) Ceiling sample used to generate ceiling texture

Figure 12: Example samples for floor and ceiling texturing. Refer to Fig. 13 and Fig. 14 for the generated textures.

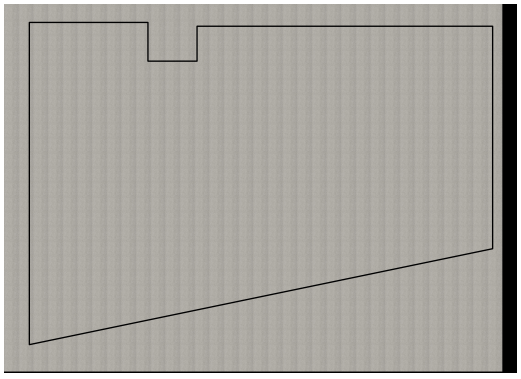


Figure 13: Example of a floor texture generated from the sample depicted in Fig. 12a. The layout of the room is delineated by solid black lines.

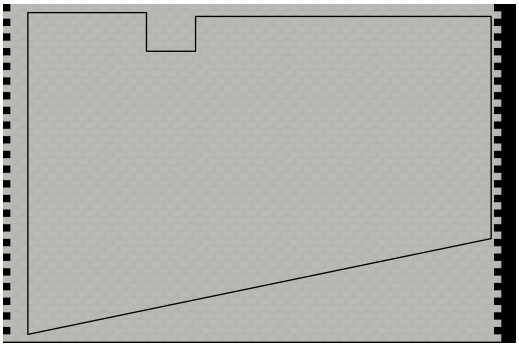


Figure 14: Example of a ceiling texture generated from the sample depicted in Fig. 12b. The layout of the room is delineated by solid black lines.

7.3 Extension of RoomRecon to MultiRoom

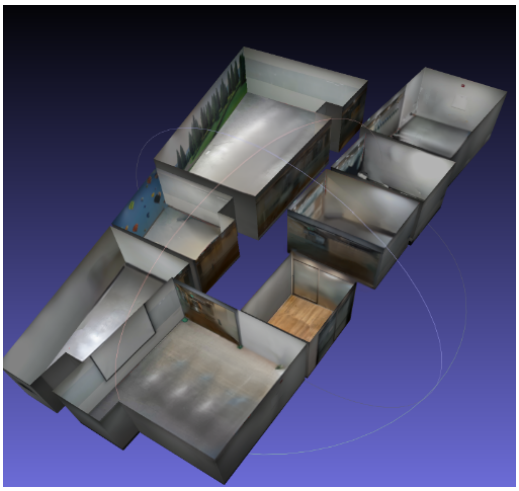


Figure 15: Extension of RoomRecon to MultiRoom.

In this paper, our primary goal is to introduce and evaluate the RoomRecon pipeline in the context of single-room reconstruction. However, the versatility of RoomRecon extends seamlessly to multi-room environments. By using RoomRecon to reconstruct rooms one by one, we can construct comprehensive multi-room environments that span entire floors, such as office spaces or apartments.

The RoomRecon pipeline, when applied to a multi-room environment, produces the result shown in Fig. 15. This extension is straightforward, provided that the respective room layouts and camera images (specifically, camera extrinsics) adhere to a consistent world coordinate system.

This extension greatly expands the scope of the application by facilitating the creation of realistic and fully textured interiors over large areas. Its impact spans multiple domains, including augmented reality (AR), virtual reality (VR), robotics, retail and real estate.

7.4 Further visualizations of the Texturing step

In Fig. 16, we illustrate the occlusion test, a crucial component of assignment-based texturing methods [15, 37]. This test is crucial for filtering candidate texture images for each mesh face during the Texturing step (Section 3.6). For a given image-face pair, successfully projecting three vertices of a face onto an image designates it as a candidate texture image for that mesh face. Conversely, if any ray connecting each vertex to a camera center is obstructed by other parts of the mesh, the image is eliminated as a candidate.

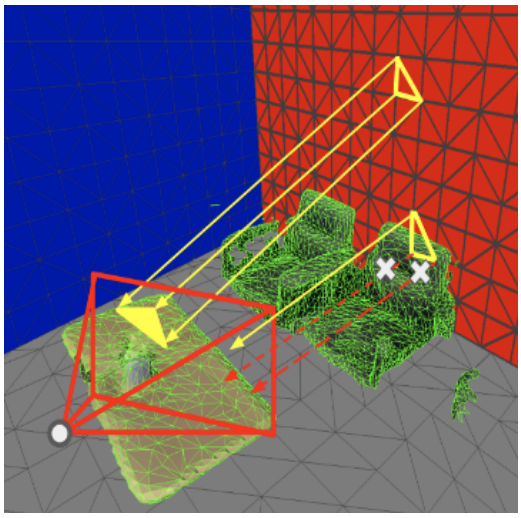


Figure 16: Occlusion test used in texturing methods [15, 37] for an image and a face pair.

MVSTex [37] and SeamlessTex [15] employ additional refinement techniques, including back-face culling and angle criteria, to enhance the selection of candidate images. Specifically, the angle between a ray from the face center to the camera center, and the face normal should not exceed a certain angular threshold - e.g., 45 degrees. The face-image qualities are then stored in a sparse two-dimensional table, with rows representing images and columns representing faces. This table stores unary costs that represent the scores of the face-image pairs. During the Markov Random Field (MRF) optimization process for texture mapping, this table is used to determine the optimal assignment of texture images to mesh faces. In addition, pairwise costs are included in the MRF optimization to enforce the use of the same texture image for neighboring faces.

In our problem setting of creating a complete and high-fidelity 3D textured room layout, several technological and environmental challenges arise during the Texturing step (Section 3.6) that utilizes the assignment-based texturing methods [15, 37]. These problems are grouped into eight distinct categories (A-H) as visualized in Fig. 21.

A: Geometry and camera misalignments often occur due to incomplete and inaccurate meshes, as well as camera pose drift in VIO (Visual-Inertial Odometry) pose tracking systems [4]. An example of this is shown in Wall 6, where the chair textures invade the wall texture due to misaligned camera poses and incomplete chair meshes. Although the intent of the occlusion test (Fig. 16) is to exclude this image from texturing that specific part of the plane, the incomplete chair meshes result in its inclusion during the Texturing process [15, 37]. Wall 4 also exhibits this issue, characterized by the texture of the refrigerator intruding into its surface texture.

B: Texture seams are caused by camera pose drift. Assignment-based texturing methods use multiple images to texture a single planar structure. If these images have misaligned poses, noticeable texture seams result. See Wall 6 in Fig. 21.

C: Uncaptured areas and **D**: Occlusion present similar challenges, but from different sources. Uncaptured areas, such as those on Wall 1 and Wall 2, occur when users fail to capture certain portions of a wall. Conversely, occlusion problems occur when objects obstruct planar surfaces (Wall 3, Wall 4, Wall5, Wall 6), particularly floors due to many furniture on them.

E: The blending of object shadows onto wall textures, as shown in Wall 2, is an issue that is influenced by object placement and lighting conditions.

F: The thin-object problem occurs when slender structures are not fully captured during the initial Capturing step (Section 3.3), often due to depth sensor resolution limitations. In such cases, the mesh of thin objects is incomplete, leading to an erroneous result in the occlusion test (Fig. 16). Note that Wall 1 has a texture of clothes hanger on its surface texture.

G and **H**: Reflective and transparent surfaces introduce additional complexities. For example, reflective surfaces such as mirrors and whiteboards produce unwanted reflections, while transparent walls, such as glass walls, lead to inconsistent viewpoints (e.g., inconsistent views of outdoor scenes) from images used to texture a planar wall. Look at Wall 5 and 6 for these problems.
Ceilings and floors, which typically consist of repeating patterns, are particularly susceptible to camera pose drift. Even small discrepancies caused by pose drift result in conspicuous misaligned lines that significantly compromise visual aesthetics.

7.5 Further visualization of inpainting and stable-diffusion

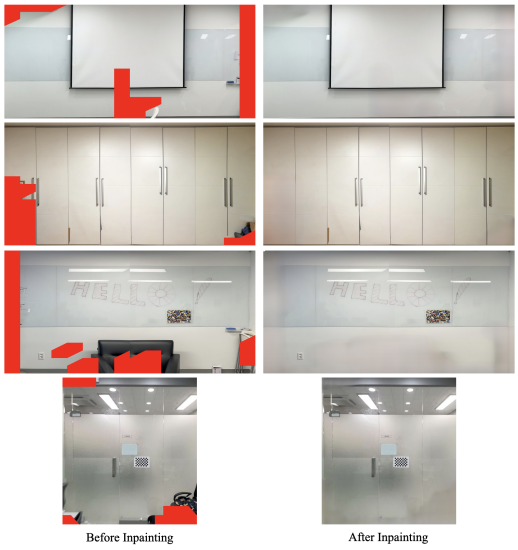


Figure 17: Examples of plane images before and after inpainting: We use ZIT [13] as our inpainting model. Red masks are untextured areas caused by occlusion by furniture.

In Fig. 17, we present examples of images before and after inpainting with ZIT [13], demonstrating the effectiveness of inpainting in room layout reconstruction applications. While any inpainting method could be used for the Post-Texturing (Sec. 3.8), we specifically chose ZIT for the following reasons. ZIT uses the Transformer Structure Restoration module to restore missing structures such as edges and lines. This is particularly useful for interior scenes, which are constituted by a lot of boundary patterns (e.g. doors, windows and paintings). Furthermore, in the qualitative comparison, ZIT proves its suitability for the task of seamlessly filling in masked areas, as shown in Fig. 19. Taking the red box in the first row of Fig. 19 as an example, LAMA fails to recover the different colours of the board and wall area. Also, MAT tends to replace the masked area with another object. In contrast, ZIT effectively produces a high-quality image, successfully reconstructing the masked areas with the matching colours of different objects. It also creates a crisp boundary between them. Moreover, ZIT shows benefits of reducing blurry prediction often

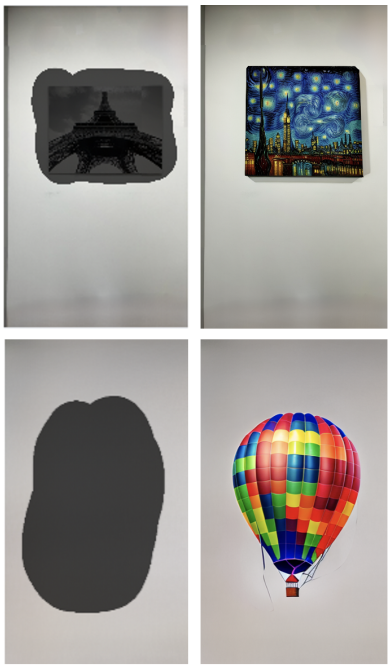


Figure 18: Examples of the Post Texturing (ImageMode) results using the stable diffusion model (PowerPaint [48]), which is a server-based service on RoomRecon. In the first example (the first row), we input the text *"A painting, hanging on the wall, portrays a city at night in the style of Van Gogh"*. In the second example (the second row), the input text is *"a flying balloon"*.

produced by LAMA and MAT (orange boxes). The final reason for choosing ZIT is the active update from the author. Recently, an extended version of ZIT (ZIT++ [8]) is released, providing the improvement in both performance and run time. A head-to-head quantitative comparison between LAMA, MAT, and ZIT as well as information of ZIT++ can be found in the paper of ZIT++ [8]. It is worth noting that other inpainting methods can be easily integrated in RoomRecon, based on the user's requirements. For example, LAMA inpainting is faster than ZIT, reducing waiting time for users, but yields lower inpainting quality.

Furthermore, Fig. 18 demonstrates the application of the Stable Diffusion model [48] in image processing, facilitating the customization and personalization of wall textures to meet diverse user preferences.

7.6 Ground truth image samples for local texture quality assessment

For the local texture quality assessment (Sec. 4.2), the ground truth images were captured of areas with significant detail while avoiding regions with uniform textures such as plain white walls. We focused on areas such as door and window frames, wall paintings, and edges, where texture seams are highly noticeable and can lead to unsatisfactory results. This approach allowed us to effectively compare different texturing methods in these critical regions. Fig. 22 shows three representative samples of the ground truth images for each room. It can be seen that our objective was to capture areas with a high degree of detail with many lines/edges and colors.

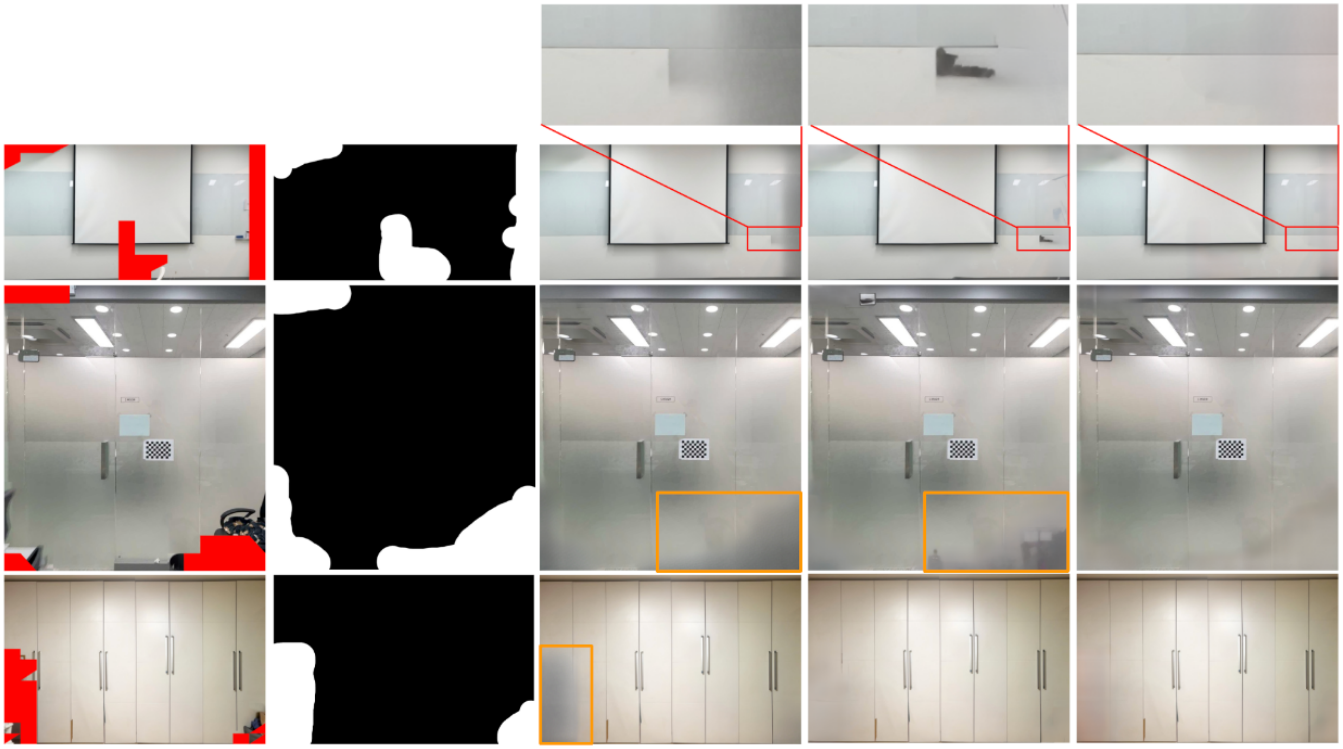


Figure 19: Visualisation of images inpainted by various methods, including LAMA [35], MAT [25] and ZIT [13]. Red areas indicate untextured areas caused by occlusion of furniture and inpainted using the inpainting methods.

7.7 Further visualization of textured rooms

Fig. 20 presents additional views of 3D textured models for five rooms discussed in the main paper (Fig. 9 of the paper), offering a comprehensive depiction of each textured room.
Next, Fig. 23 illustrates textured room layouts for 7 different rooms. The left column shows the room layouts, while columns two and three show textured rooms, viewed from two different perspectives. Columns four and five show samples used for floor and ceiling textures, created using the SampleMode of the Post Texturing step (Section 3.8).
Finally, Fig. 24 and Fig. 25 display three textured walls from each room, illustrating the high texturing quality achievable with the introduced pipeline.

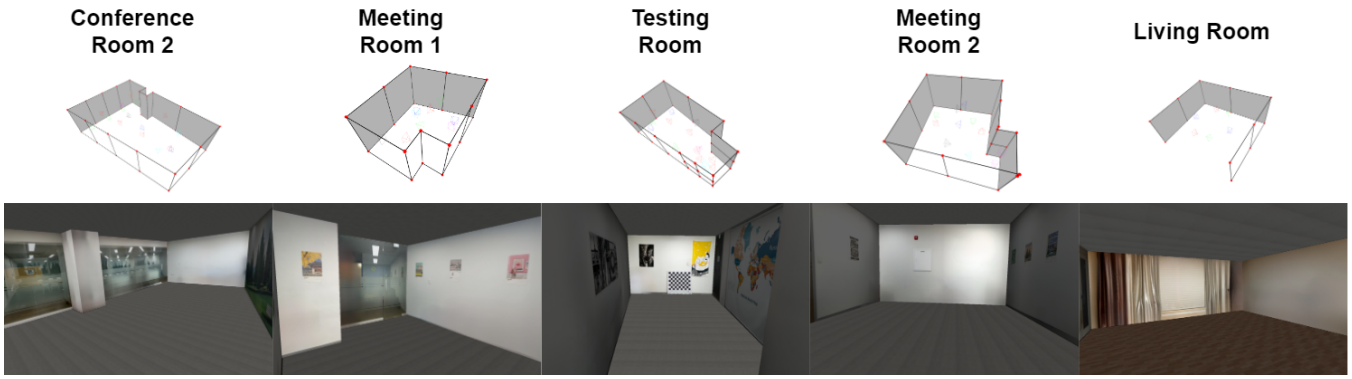


Figure 20: Additional views of five rooms from the paper.

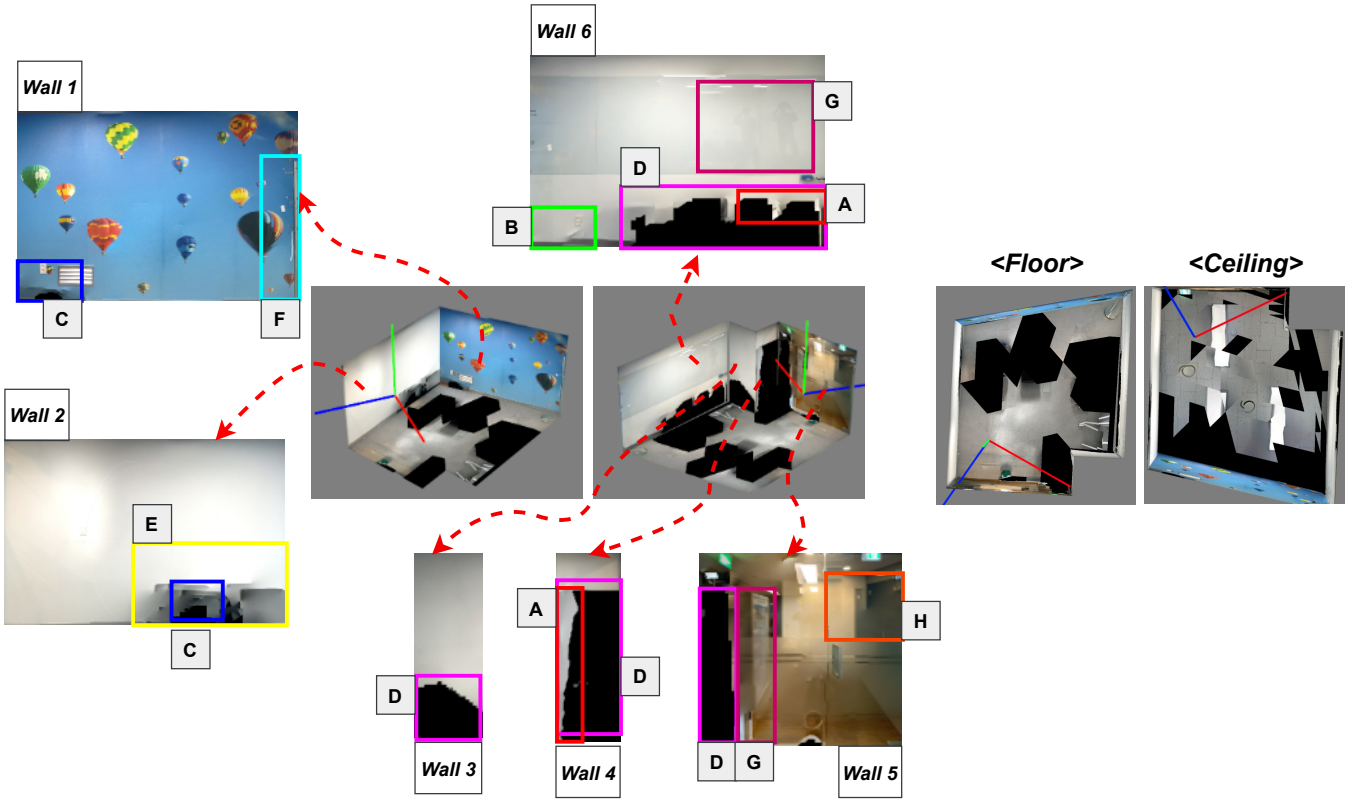


Figure 21: Texturing problems that occur in a room - A: Geometry and camera misalignment, B: Texture seams, C: Uncaptured areas, D: Occlusion, E: Object shadows, F: Thin objects, G: Reflection, H: Different views.

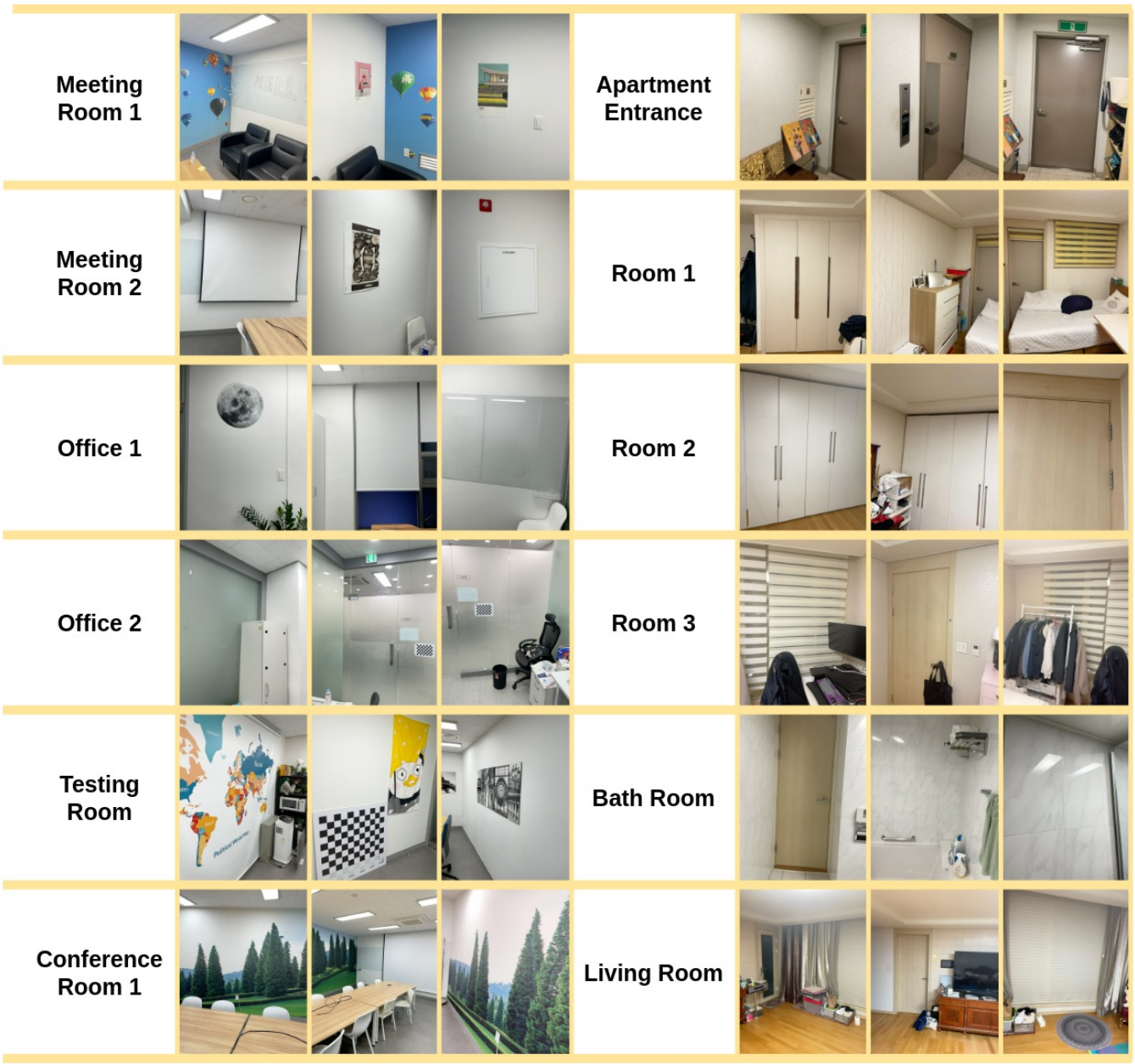


Figure 22: Representative samples of the ground truth images captured for each room.

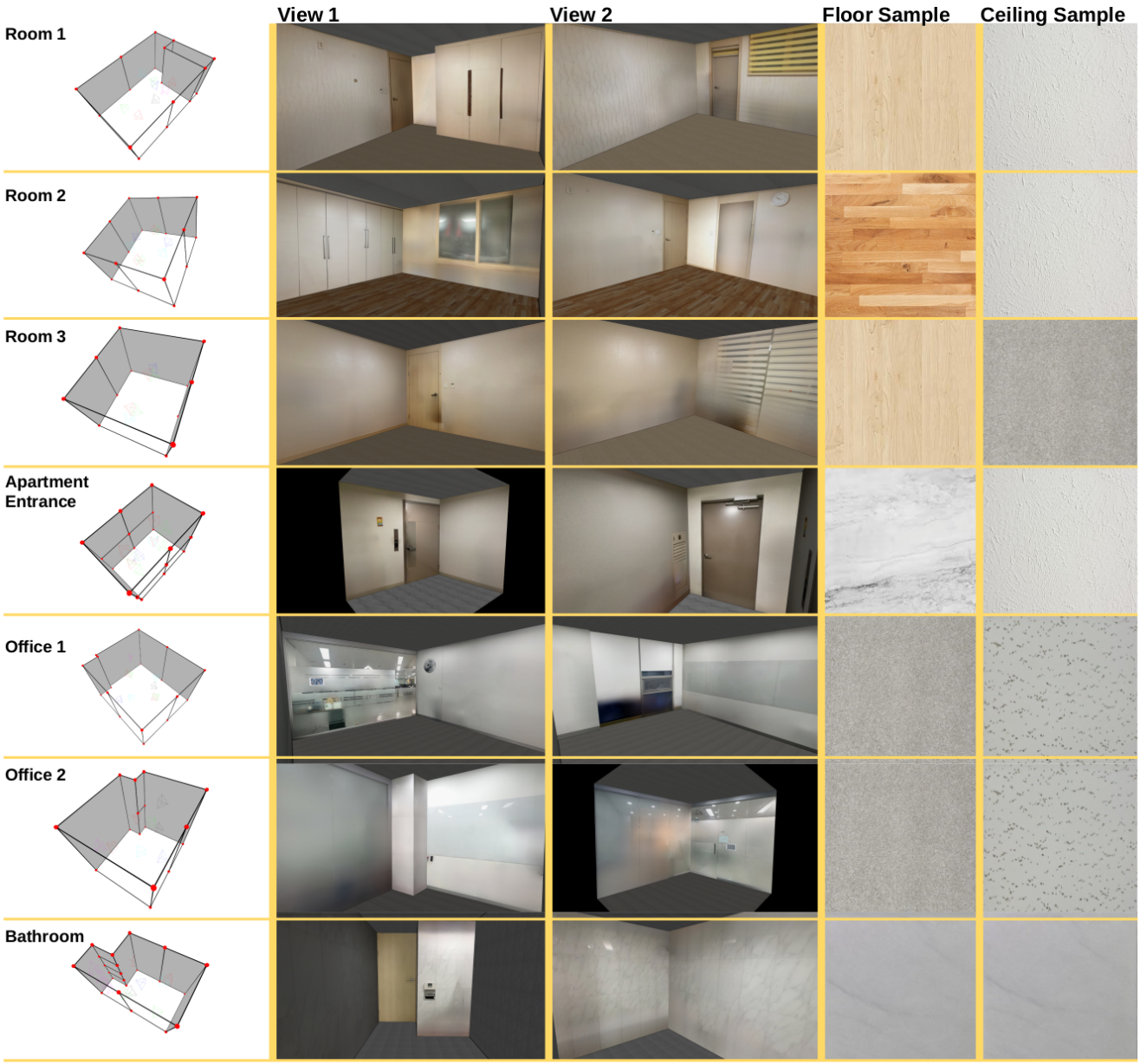


Figure 23: Visualization of textured rooms: The left column shows the room layouts, while the 2nd and 3rd columns show textured room layouts seen from two different perspectives. The 4th and 5th columns show samples used for the generation of floor and ceiling textures.

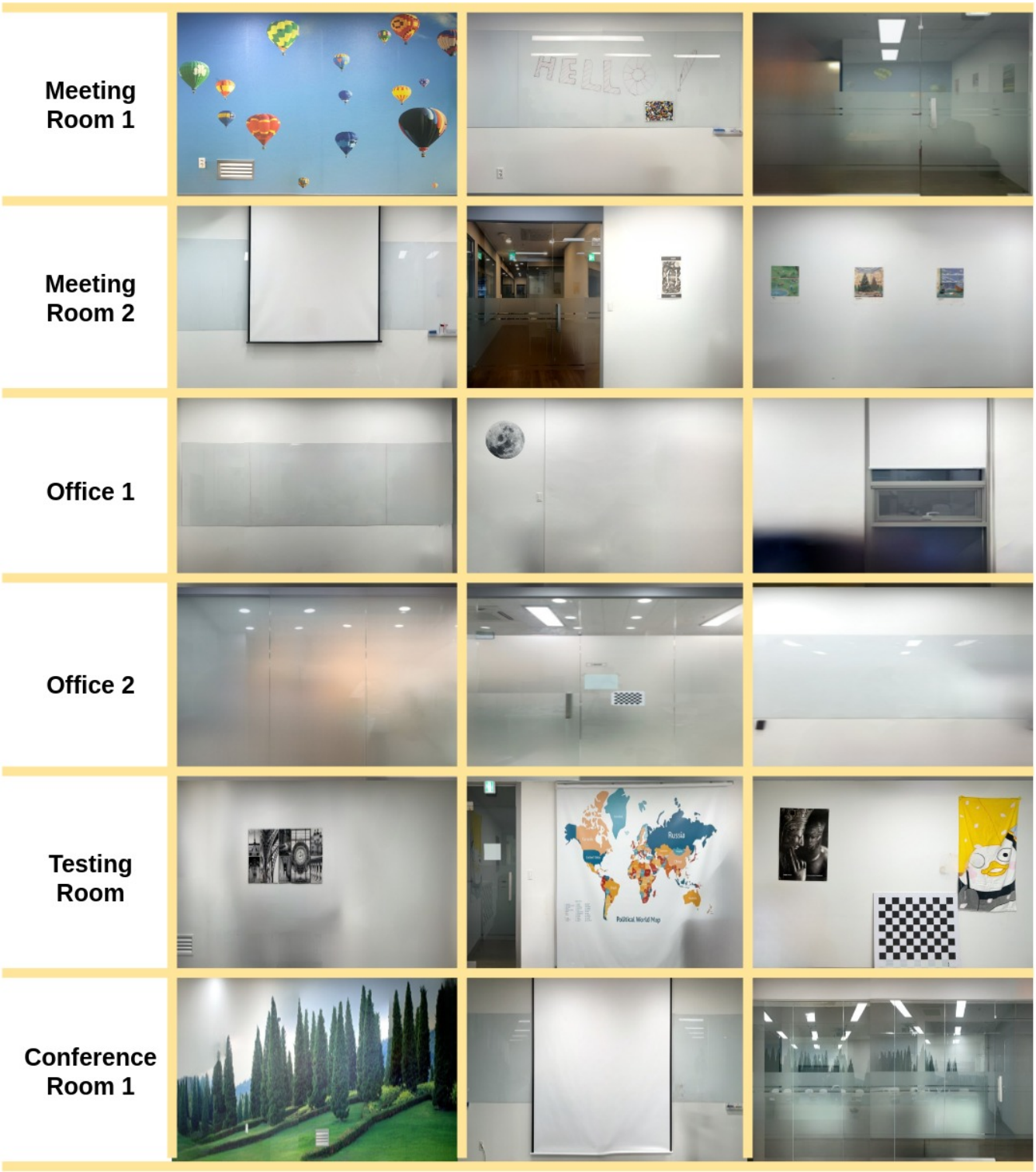


Figure 24: Additional visualizations of the textured walls for each room after the inpainting process - 1/2.

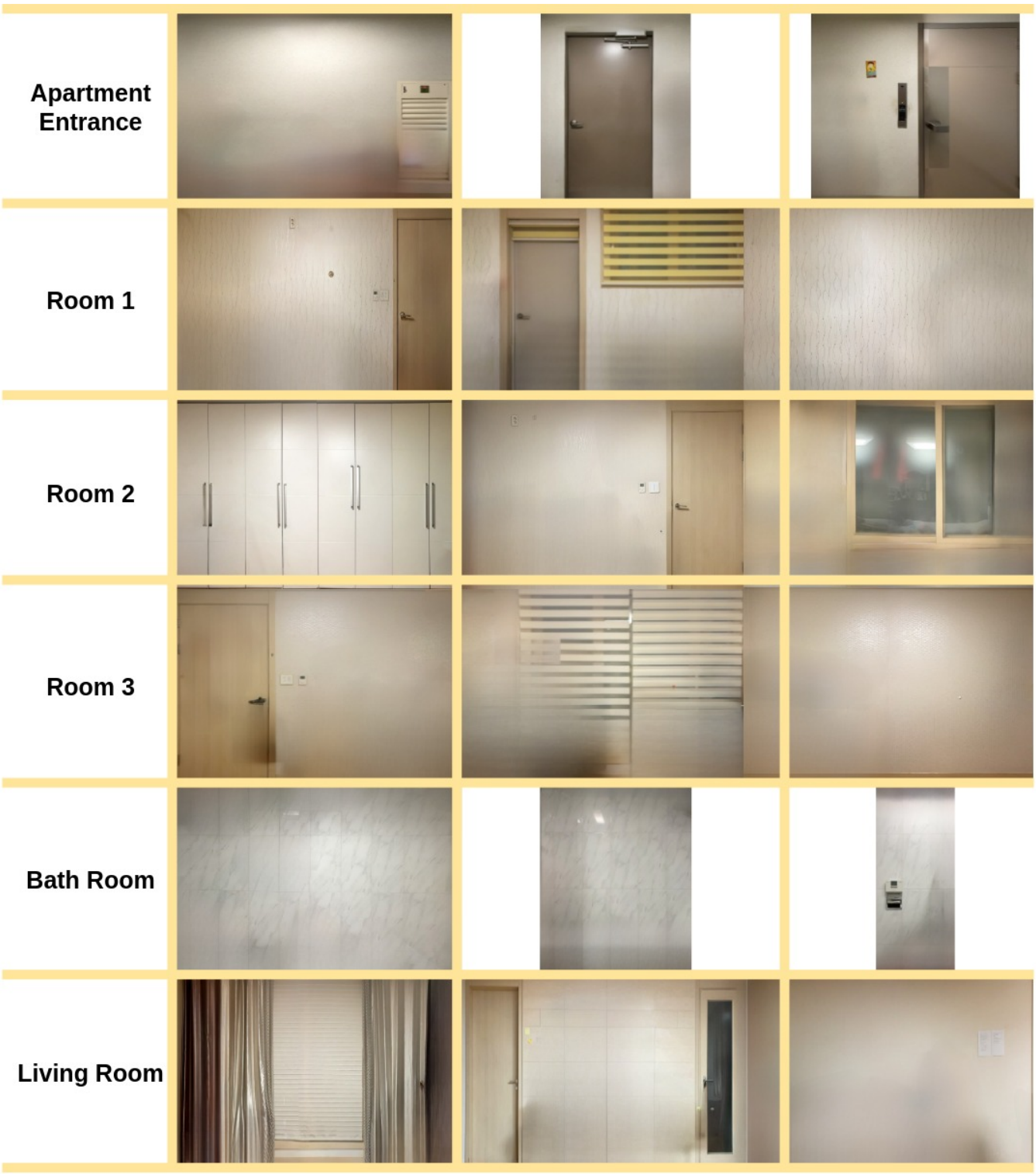


Figure 25: Additional visualizations of the textured walls for each room after the inpainting process - 2/2.

8 ALGORITHMS

In this section, we present the pseudocode for algorithms discussed in the main paper. Initially, Algorithm 1 details the Plane2Image rendering process, as described in Section 3.7. Following this, Algorithm 2 elaborates on the computation of the perspective and view matrices essential for the rendering process. Next, Algorithm 3 introduces the "divide" component of our Divide-And-Conquer scheme, which is covered in Section 3.5. This component focuses on determining optimal camera positions for capturing texture images of sub(planes).

8.1 Plane2Image algorithm

Algorithm 1 Plane2Image algorithm

function PLANE2IMAGE($planeIdx, isVertical, isFloor$)
  if $isVertical$ **then** ▷ Check if it is a vertical plane
    $vertiInfo \leftarrow$ planeInfo[$planeIdx$]
    $planeNormal \leftarrow$ vertiInfo.planeNormal
    $planeCenter \leftarrow$ vertiInfo.planeCenter
    $planeWidth \leftarrow$ vertiInfo.planeWidth
    $planeHeight \leftarrow$ vertiInfo.planeHeight
  else
    $mbbCenter2D \leftarrow MBB_{center}$
    $planeWidth \leftarrow MBB_{width}$
    $planeHeight \leftarrow MBB_{height}$
    if $isFloor$ **then**
      $planeNormal \leftarrow (0,1,0)$
      $planeCenter \leftarrow (mbbCenter2D.x, \text{floorHeight}, \ldots$
        $mbbCenter2D.y)$
    else
      $planeNormal \leftarrow (0,-1,0)$
      $planeCenter \leftarrow (mbbCenter2D.x, \text{ceilingHeight}, \ldots$
        $mbbCenter2D.y)$
    end if
  end if
  $viewMatrix, projectionMatrix \leftarrow$ SETUPCAM($planeCenter, planeNormal, planeWidth, planeHeight, isFloor$)
  $RENDER(viewMatrix, projectionMatrix)$
end function

Algorithm 2 Set up a camera for plane rendering for the Plane2Image operation

function SETUPCAM($center, normal, width, height, isVertical, isFloor$)
  $perMeterPixelCount \leftarrow 500$ ▷ Defines the resolution in pixels per meter.
  $lookAtVector \leftarrow center$
  $cameraPosition \leftarrow center + normal \times height$
  $upVector \leftarrow (0,1,0)$ ▷ Default up vector
  if Not $isVertical$ **then**
    if $isFloor$ **then**
      $upVector \leftarrow -\mathbf{MBB}_{\text{align}}$
    else
      $upVector \leftarrow \mathbf{MBB}_{\text{align}}$
    end if
  end if
  $viewMatrix \leftarrow$ calculateViewMatrix($cameraPosition, lookAtVector, upVector$)
  $planePixelWidth \leftarrow \text{int}(perMeterPixelCount \times width)$
  $planePixelHeight \leftarrow \text{int}(perMeterPixelCount \times height)$
  $near \leftarrow 0.001f$ ▷ Near clipping plane for rendering
  $far \leftarrow 100.0f$ ▷ Far clipping plane for rendering
  $fx \leftarrow planePixelHeight/2.0$
  $fy \leftarrow planePixelHeight/2.0$
  $cx \leftarrow planePixelWidth/2.0$
  $cy \leftarrow planePixelHeight/2.0$
  $imageWidth \leftarrow planePixelWidth$
  $imageHeight \leftarrow planePixelHeight$
  $projectionMatrix \leftarrow$ calculateProjectionMatrix($planePixelWidth, planePixelHeight, fx, fy, cx, cy, near, far$)
  return $viewMatrix, projectionMatrix$
end function

8.2 Divide-And-Conquer algorithm

Algorithm 3 Divide-And-Conquer algorithm for Texture Image Acquisition

function DIVIDEANDCONQUER($planeIdx, walls$)
  $info \leftarrow walls[planeIdx]$
  $normal \leftarrow info.planeNormal$ ▷ Plane normal
  $center \leftarrow info.planeCenter$ ▷ Plane center
  $width \leftarrow info.planeWidth$ ▷ Plane width
  $height \leftarrow info.planeHeight$ ▷ Plane height
  $fov \leftarrow$ Camera's field of view
  $optPosition \leftarrow center + normal \times$ optimal distance
  $canCapture \leftarrow$ CANCAPTURE($width, height, fov, optPosition$)
  if *canCapture* **and** WITHINWALLS(optPosition, walls) **and** optimal distance ≤ 5 **then**
    return $optPosition$
  else
    $subPlanes \leftarrow$ DIVIDEPLANE($width, height, 2$)
    for sp in *subPlanes* **do**
      return DIVIDEANDCONQUER($sp, walls$) ▷ Recursion until the conditions are met
    end for
  end if
end function

function CANCAPTURE(w, h, fov, pos)
  return $(w \leq fov.width)$ and $(h \leq fov.height)$
end function

function WITHINWALLS($pos, walls$)
  return True if *pos* is inside the polygon formed by *walls*
end function

function DIVIDEPLANE($w, h, factor$)
  $newW \leftarrow w/factor$
  $newH \leftarrow h/factor$
  return $[(newW, newH), (newW, newH)]$ ▷ Splitted to two subplanes
end function

9 QUANTITATIVE STATISTICS OF EXPERIMENTAL DATASETS

In this section, we present the comprehensive results of the experiments described in the main paper. It is important to note that the numbers given in the main paper represent averages, which are also provided at the bottom of each table for clarity and reference.
Tab. 5 presents the processing time measurements for each step of the RoomRecon pipeline on an iOS device, specifically an iPhone 12 Pro with an A14 Bionic chip, excluding texturing-related processes. The time measurements for the texturing-related processes are detailed in Tab. 6.
Tab. 5 outlines the following processes:

1. Layout Parsing: This step involves acquiring layout information, including the normals, centers, and four vertices of the vertical walls, as well as the floor and ceiling heights.

2. Loop Check and Form Floor and Ceiling: Loop Check step verifies whether the vertical walls form a closed loop, essential for subsequent mesh processing steps. Using this information, floor and ceiling meshes are generated using the Constrained Delaunay Triangulation (CDT) method [3] or the Minimum Bounding Box (MBB) [2] as described in Section 3.4 of the main paper.

3. Mesh Processing (Section 3.4): This process includes mesh filtering, remeshing, and combining to produce the combined mesh (M_{combined}).

4. Plane2Image (Section 3.7): This step involves rendering the textured meshes using our Plane2Image module.

5. Displaying Results: Finally, the textured meshes (M_{textured}) are added to the scene for visualization and further Post Texturing processes (Section 3.8).

Tab. 6 presents a comparative analysis of the computational time required for texturing-related processes on an iOS device, an iPhone 12 Pro with an A14 Bionic chip, before and after the implementation of our Divide-And-Conquer scheme. The texturing method used in this experiment, MVSTex [37], initially involved calculating the blurriness of each image to sample sharp images from $C_{original}$, represented as Blur Calculation Time. However, our Divide-And-Conquer approach eliminates the need to compute blurriness because all captured images ($C_{recaptured}$) are consistently sharp. The computational overhead for the "divide" operation (Division Calculation Time) is minimal. By using

a significantly reduced number of images (from 258.85 to 11.39), the texturing process using $C_{recaptured}$ (Our-MVSTex) is accelerated, now completing in only 4.72 seconds on an iOS device.

Tab. 7 provides a quantitative analysis of the number of frames (Total Frames, Sampled Frames, Recaptured Frames, Ground truth (GT) Frames) and their blur measures. Our recaptured images show sharper characteristics than other image groups (Total Frames, Sampled Frames) in all rooms. Blur metric used in this experiment is from the work of Crete et al. [11].

Tab. 8 presents a texturing time analysis of all texturing methods [15, 37, 38, 46] used in the experiment on a Linux computer equipped with an AMD Ryzen 9 5900x 12-core × 24 processor. The GPU specifications of the machine are not reported because none of the texturing methods involve GPU operations.

	Layout Parsing [ms]	**Loop Check [ms]**	**Form Floor / Ceiling [ms]**	**Mesh Processing [ms]**	**Plane2Image [ms]**	**Displaying Results [ms]**	**Total [ms]**
Meeting Room 1	3.291	0.044	2.229	239.042	4291.335	476.295	5012.236
Meeting Room 2	5.082	0.028	2.195	344.215	4550.166	751.736	5653.422
Office 1	4.378	0.051	2.299	115.617	3798.089	382.080	4302.513
Office 2	5.342	0.049	2.264	265.978	4498.490	566.005	5338.128
Testing Room	3.506	0.029	2.525	339.635	5597.870	632.177	6575.742
Conference Room 1	3.823	0.016	2.133	219.718	3506.317	1212.163	4944.170
Conference Room 2	6.304	0.033	3.466	693.724	7042.335	1449.250	9195.112
Apartment Entrance	3.125	0.017	1.353	43.225	2643.013	276.860	2967.593
Room 1	3.412	0.115	2.202	197.832	4843.752	466.239	5513.551
Room 2	1.915	0.017	1.411	124.408	1690.656	410.992	2229.400
Room 3	2.120	0.017	1.419	114.148	3121.076	339.858	3578.637
Bath Room	3.527	0.026	2.167	78.829	3409.786	284.500	3778.835
Living Room	2.284	0.020	1.394	203.665	2063.789	613.244	2884.395
Aggregated	3.701	0.036	2.081	229.234	3927.436	604.723	4767.210
Std Dev.	±1.298	±0.027	±0.588	±168.003	±1455.371	±354.626	±1836.356

Table 5: Detailed timing analysis for each step of RoomRecon on iOS devices.

	Number of original frames ($C_{orig.}$) [-]	**Blur Calculation Time [ms]**	**Number of selected frames [-]**	**Texturing Time-Before [ms]**	**Division Calculation Time [ms]**	**Number of recaptured frames ($C_{recaptured}$) [-]**	**Texturing Time-Our [ms]**
Meeting Room 1	919	7420.991	405	23307.680	0.214	9	4911.648
Meeting Room 2	503	4038.300	233	22292.851	0.239	14	4960.514
Office 1	528	4342.993	230	13908.482	0.228	9	3179.689
Office 2	380	3144.345	179	21819.396	0.207	16	6263.140
Testing Room	1148	9530.300	471	34307.223	0.296	17	6654.108
Conference Room 1	779	6539.145	327	25441.619	0.259	8	4293.619
Conference Room 2	865	7317.129	370	34061.953	0.284	18	9489.955
Apartment Entrance	481	3816.633	200	7705.158	0.231	9	1928.829
Room 1	431	3306.680	185	13008.757	0.242	14	4440.505
Room 2	273	2091.293	129	7491.302	0.175	8	3414.745
Room 3	680	5267.885	290	19219.999	0.190	8	4186.139
Bath Room	282	2128.644	125	7481.924	0.228	10	1804.568
Living Room	530	4162.328	221	13170.632	0.188	8	5785.215
Aggregated	599.923	4854.359	258.846	18708.998	0.229	11.385	4716.360
Std Dev.	±262.900	±2243.389	±106.981	±9253.167	±0.036	±3.820	±2061.859

Table 6: Comparative analysis of texturing times before and after the introduction of **Divide And Conquer** on iOS devices.

Table 7: Quantitative analysis of number of frames (Total Frames, Sampled Frames, Recaptured Frames, GT Frames) and blur metrics (Total Blur, Sampled Blur, Recaptured Blur) across rooms

Room Type	Total Frames	Sampled Frames	Recaptured Frames	GT Frames	Total Blur ↓	Sampled Blur ↓	Recaptured Blur ↓
Meeting Room 1	919	405	9	20	0.38	0.34	0.23
Meeting Room 2	503	233	14	15	0.37	0.34	0.22
Office 1	528	230	9	14	0.37	0.35	0.24
Office 2	380	179	16	15	0.37	0.33	0.25
Testing Room	1148	471	17	16	0.33	0.31	0.21
Conference Room 1	779	327	8	17	0.35	0.32	0.23
Conference Room 2	865	370	18	20	0.36	0.34	0.26
Apartment Entrance	481	200	9	12	0.44	0.40	0.25
Room 1	431	185	14	13	0.50	0.46	0.27
Room 2	273	129	8	15	0.47	0.43	0.29
Room 3	680	290	8	10	0.49	0.46	0.26
Bath Room	282	125	10	14	0.43	0.39	0.25
Living Room	530	221	8	20	0.44	0.41	0.28
Aggregated	599.92	258.85	11.38	15.46	0.41	0.37	0.25
Std Dev	±262.90	±106.98	±3.82	±3.13	±0.06	±0.05	±0.02

Table 8: Comparison of time metrics for different texture methods (performed on Linux machine).

Room Type	ColorMapOpt [46] (ms)	PlaneOpt [38] (ms)	MVSTex [37] (ms)	SeamlessTex [15] (ms)	Our-MVSTex (ms)	Our-SeamlessTex (ms)
Meeting Room 1	115352	204867	12867	43502	3163	3912
Meeting Room 2	61712	347866	7692	25101	2555	3885
Office 1	70766	109676	8050	28356	2941	3862
Office 2	98909	197003	7218	20941	4526	5937
Testing Room	137755	290077	7979	87402	4179	5963
Conference Room 1	62039	684336	8908	36407	2418	3293
Conference Room 2	140178	521584	10357	41069	5246	6665
Apartment Entrance	50564	125640	7235	22719	1977	2989
Room 1	70610	104626	7413	21395	3793	5766
Room 2	27317	373976	7049	14887	2755	4044
Room 3	65555	211477	9191	34844	2832	3956
Bath Room	36651	55988	5108	14156	1600	2535
Living Room	83148	332202	11178	33278	3960	4972
Aggregated[s]	78.50	273.79	8.48	32.62	3.23	4.44
Std Dev[s]	±35.48	±179.96	±2.03	±18.93	±1.05	±1.29